slides.com/jimmckeeth/decompile-debug-bsdg

github.com/jimmckeeth/decompile-debug-bsdg

BSDG.dev
Jan 13, 2026
Jim McKeeth
jim@mckeeth.org
delphi.org

Decompile & Debug

Exploring the relationship between the high level source code we write an the binary machine code that powers our computers.

Blah, Blah, Blah

- Presentation by Jim McKeeth
- New & original materials licensed under Creative Commons BY-NC-SA 4.0
- Research pulled from experience and various source, including product and project pages, 3rd party publications, and large language models
- Independently verified as possible, considered biased opinion otherwise

- Icons: Icons 8 | Noun Project | Font Awesome | Google
- Graphics: Pixabay | Pexels | Unsplash
- Images & reference: Wikipedia | Wikimedia
- LLM Research & Infographics: Google Gemini
- Image Editing: Midjourney | banana.ovo.re | Gimp
- Slides: Slides.com | Reveal.js

slides.com/jimmckeeth/decompile-debug-bsdg
github.com/jimmckeeth/decompile-debug-bsdg


Why?

THE LAW OF LEAKY ABSTRACTIONS:

HIGH-LEVEL ABSTRACTION (Ideal)

Looks Perfect (e.g., 'Write Once, Run Anywhere')

THE LEAK (Abstraction Fails)

UNDERLYING COMPLEXITY SEEPS THROUGH

MOV EAX, EBX

0x004F5A

0x004F5A

SEGNENTATION FAULT

0x0016

KERNEL PANIC

THE LEAK (Abstraction Fails)

TO OPERATE EFFECTIVELY UP HERE, YOU MUST UNDERSTAND DOWN THERE.

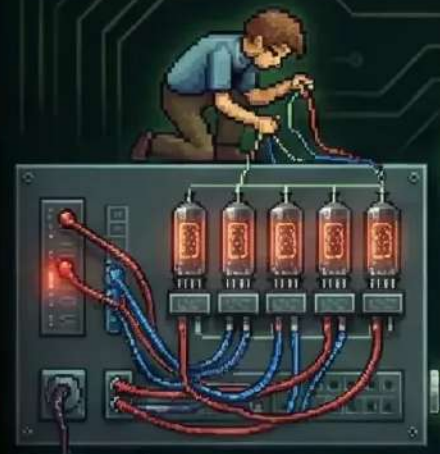
LOW-LEVEL REALITY (Understanding Required)

Memory Management, Network Latency, File System Locks, Driver Crashes, Hardware Interrupts.

GAME OVER

EFFECTIVE HIGH-LEVEL OPERATION *REQUIRES* LOW-LEVEL UNDERSTANDING.

EVOLUTION OF PROGRAMMING: THE ABSTRACTION ASCENSION



HARDWARE LEVEL: MACHINE CODE

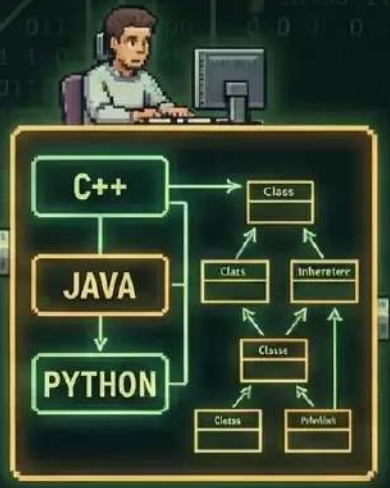
Direct Control.
Zero Abstraction.
Wires & Switches.

ASSEMBLY LANGUAGE (LOW-LEVEL)

Mnemonics.
1:1 Mapping to CPU Instructions.
Near Metal.

COMPILED LANGUAGES (HIGH-LEVEL)

Procedural Logic.
Memory Management.
Portable.
Abstracted from Hardware.



OBJECT-ORIENTED & VIRTUAL MACHINES

Real-world Modeling.
Encapsulation.
Interpreter/VM Layer.
Further Detachment.

SCRIPTING & INTERPRETED (HIGHEST ABSTRACTION)

Rapid Dev. Dynamic.
Interpreted on the Fly.
Far from Metal.

ABSTRACTION LEVEL

LOW
(NEAR METAL)

HIGH
(FAR FROM METAL)


MS
DOS


DEBUG.COM

Debug.exe

Originally written by Tim Paterson for 86-DOS

Microsoft hired him & added it to MS-DOS

Still available through Windows XP

Also in FreeDOS, DR-DOS, and other clones.

en.wikipedia.org/wiki/Debug_(command)

- The original - launched in 2002
- Focused on game emulation

- Emulates 286/386/486 real and protected modes
- Also emulates relevant hardware
- The defacto standard for DOS emulation on *many platforms*

- Development slowed in late 2010s
- Last major release 0.74-3 is from 2019

- dosbox.com
- sourceforge.net/projects/dosbox

Alternatives

dosbox-staging.org

dosbox-x.com

github.com/schellingb/dosbox-pure

See Also: vdos.info (DOS line of business app focus)

github.com/MartyShepard/DOSBox-Optionals (Win 95 gaming)

DOSBox Staging

dosbox-staging.org

- Latest Release 0.82.2 on Jun 17, 2025
- Focus:
  - High-fidelity Gaming
  - Modern Hardware
  - "Drop-in" Replacement for DosBox
- Dropped support for legacy OS (Windows 7 or older)
- Includes authentic adaptive CRT emulation
- Sensible Defaults

DosBox Pure

- Focused on Simplicity & Ease of Use
- Latest release Pure 1.0 Preview 4 from Nov 2025
- Built for RetroArch/Libretro (game emulator frontends)
- Save states with automatic rewind
- Adds controller support

github.com/schellingb/dosbox-pure

DOSBox-X

dosbox-x.com

- Focused on completeness
- A general-purpose x86 emulator that also supports gaming
- Latest Release: 2026.01.02 (Jan 2, 2026)
- GUI and Menu Bar configuration
- Clipboard support
- Good documentation and guides
  - dosbox-x.com/wiki/
  - dosbox-x.com/wiki/Guide:Installing-FreeDOS
  - dosbox-x.com/wiki/Guide:Clipboard-support-in-DOSBox-X
  - dosbox-x.com/wiki/DOSBox-X's-Feature-Highlights

	DOSBox Staging	DOSBox-X	DOSBox Pure	vDos	DOSBox Optionals
Primary Focus	Modern Gaming/Accuracy	Complete System Emulation	Ease of Use/Controller	Business Apps	Win-Integrated Gaming
Codebase	Modern C++ (Clean)	Legacy C++ (Extensive)	Custom / Libretro base	Custom	ECE + Tweaks
Host OS Support	Win/Mac/Linux (Modern)	Win/Mac/Linux/DOS	Win/Mac/Linux	Windows Only	Windows Only
Cycle Accuracy	High (Gaming tuned)	High (Configurable)	Medium	N/A	High
Win 9x Support	Experimental	**Excellent (Drivers included)**	**Automated Install**	None	Good
PC-98 Support	No	**Yes**	No	No	No
Save States	No	**Yes** (Robust, 100 slots)	**Yes** (w/ Rewind)	No	Basic
Printing	No	**Yes** (LPT -> PDF/PNG)	No	**Yes** (Native)	No
File Locking	Basic (Risk of corruption)	Experimental	Basic	**Robust** (Win API)	Basic
Input Method	Keyboard/Mouse	Keyboard / Mouse / GUI	**Gamepad Mapper/OSK**	Keyboard	Keyboard/Mouse
Shaders/CRT	**Built-in Dynamic**	External Shaders	RetroArch Shaders	None	Integrated

VETUSWARE.COM

the biggest free abandonware downloads collection in the universe

Che Guevara never used Windows.

You: guest [login] [register]

MS-DOS books — buy link here

OS

5 recently added:	5 most popular:
DOSVAX PS17 JA	MS DOS 6.00(Sources included) (13575)
OpenVMS x86-64 V9.2-3 9.2-3	MS DOS Boot Disk 7.10 (10005)
DR DOS 5.0	OS/2 2.1 (7961)
MS-DOS 5.0 Spanish ACER 5.0 ES	MD DOS Boot Disk 7.10 (7117)
FlexOS 4.01	DOS32 Advanced 7.33 (6005)

System

5 recently added:	5 most popular:
FlexOS 286 v1.3 1.3	Visual Basic 3.0 (8637)
Doors FR	Microsoft BASIC Professional Development System 7.1 (6885)
MS-DOS 3.3 German 3.3 Release 1.2 DE	Norton Commander for Windows 2.01 (6739)
ThunderBYTE Anti-Virus for Windows 95 8.10	DOS4GW 2.01 (5570)
Renesis SVG 1.1.1 viewer Plugins 1.1.1	SoftICE Driver Suite 2.0.1 (5188)

DBMS

5 recently added:	5 most popular:
DataEase 4.5 for DOS 4.5	Borland dBASE IV 2.0 (7529)
3by5 3.0	FoxBASE 2.00 (4523)
dBase IV 4	dBASE Compiler 1.0 (3271)
Visual Objects 1.0a 1.0a	CA-Visual Objects for Windows + upgrade 1.0 (2592)
Visual Objects 1.0d Update 1.0d	Foxpro for SCO Unix 2.6 (2531)

Office

5 recently added:	5 most popular:
pc-write 1.01	WordStar 4.00 (10059)
Wordtech systems 1.1	Microsoft Word for Windows 2.0c (5751)
Adobe Acrobat Reader 8.0 8.0 PT	Microsoft Word for Windows 1 (4430)
Symphony 1.1 ES	123 4.0 (4338)
WordStar 7 ES	Assistant Series 2.00 (3926)

Utility

5 recently added:	5 most popular:
The - Handy -DOS Utilities	Disk Dupe Pro 7.0 (4749)
Know Dos Volume 1	Checkit 3.0 (2641)
Turbo Debugger v5.0 5.0	Advanced CAB Repair 1.0 (2205)
TURBO BAT 3.10	ACE, AIN, ARC, ARJ, COM, DWC, EXE, GZ, LHA, LZH, PAK, RAR, TAR, (2150)
Palert 2.4	PC Tools Deluxe 6.0 ES (2133)

Communication

5 recently added:	5 most popular:
View232 1.11	LAN Manager 2.2c (3192)
Trillian v3.1.10.0	LapLink Pro 4.00 (2427)
Antenna Calculation and Design Tools 1	LapLink 6 (1869)
KIRSCHBAUM_NETZ_153 1.53 DE	LANtastic 6.0 (1707)
DOSCIM 3.1	NeoTrace Pro 3.25 (1697)

Games

5 recently added:	5 most popular:
THE LAS VEGAS BINGO CLUB Ver 1	Zak McKracken and the Alien Mindbenders (8297)
Poffessor Erno`s Rubik`s MAGIC`CUBE ver 1	Police Quest (7937)
3 Ancient games of PIRADA	Civilization (7212)
CHEXO	Maniac Mansion (6194)
BASSTOUR 4.2	Bumpy's Arcade Fantasy (5575)

Multimedia

5 recently added:	5 most popular:
Procyon Pro 2.07 KO	Clone CD 4.2.0.2 (6818)
GS1000R Serial MIDI Driver KO	XingMPEG Player 3.02 (3947)
Allegro 1.2	PrintShop Deluxe (2289)
Turbo Trax	CorelDRAW! 2.0 (2026)
dj mix station 3 2.03	MPEGTool 1.04 (1916)

Other

5 recently added:	5 most popular:
Bartender 2.0	Mathcad PLUS 5.0 (9841)
Uninterrupted Interrupts 1.0	AutoCAD R13 R13 (8458)
Microsoft Programmer's Library 1.0 1.0	Photoshop 3.4 (5627)
LAN Manager Programmer's Toolkit (LMPTK) 2.1	3D Home Architect 1.4 (4505)
AutoCAD R14 - Patches and Utilities N/A	Mathcad 2.5 (3372)

MS-DOS 6.22

Originally 86-DOS, written by Tim Paterson of Seattle Computer Products, DOS was a rough clone of CP/M for 8086 based hardware. Microsoft purchased it and licensed it to IBM for use with Microsoft's IBM PC language products. In 1982, Microsoft began licensing DOS to other OEMs that ported it to their custom x86 hardware and IBM PC clones.

For IBM-specific releases, please see the IBM PC-DOS product page.

Available releases

1.x 2.x 3.00 3.10 3.20 3.21 3.30 3.31 4.0x 5.0 6.0 6.20 6.21

6.22 7.1 (CDU)

Release notes

Microsoft DOS 6.22 was the last standalone version from Microsoft. It was also the last from Microsoft to run on an 8088, 8086, or 286.

6.22 adds DriveSpace, a replacement for DOS 6.20's DoubleSpace drive compression that was removed in 6.21.

There's a really detailed tutorial located at http://legroom.net/howto/msdos that gives tips on how to customize DOS. We suggest you follow this tutorials suggestions for setting up and customizing DOS. However, if you're installing to a virtual machine, writing the disk images to actual floppies isn't really necessary.

Downloads

Information

Product type
OS

Vendor
Microsoft

Release date
1994

Minimum CPU
8088

Minimum RAM
512KB

Minimum free disk space
5MB

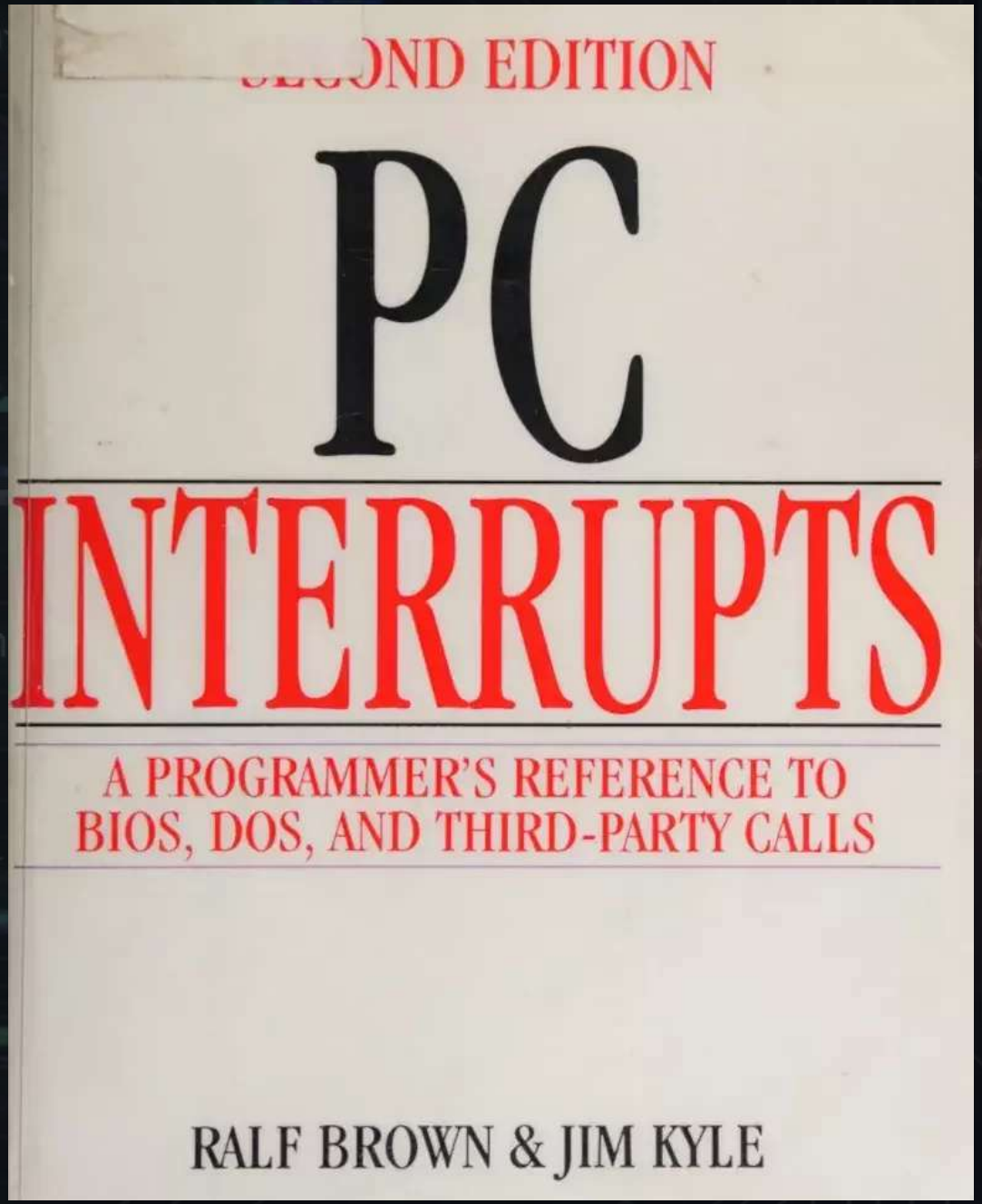
User interface
Text

Platform
DOS

Download count
1445 (570 for release)

FreeDOS

- An open source DOS-compatible operating system
- Compatible with and includes
  - Classic games
  - Legacy applications
  - Developer tools
- freedos.org - Also links to other resources
- github.com/FDOS/kernel
- gitlab.com/FreeDOS
- gitlab.com/FreeDOS/docs



Interrupts

- Ralf Brown's Interrupt List (RBIL)
  - 9600 Interrupt entries
  - 5400 tables
- Print version was 1200 pages
- Last revision 61 (17 July 2000)
  - cs.cmu.edu/~ralf/files.html (source)
  - delorie.com/djgpp/doc/rbinter
  - ctyme.com/rbrown.htm
  - fd.lod.bz/rbil

DEBUG.EXE: THE HEX EDITING ARSENAL - CRACK & PATCH GAMES

QUEST.EXE

C:\>DEBUG QUEST.EXE

LIVES: 3 (0x03)

4B2C:03

1. THE TARGET (GAME FILE)

-d 0100 L 20
-e 4B2C 90 90 90 (NOP)
-w
-q
>

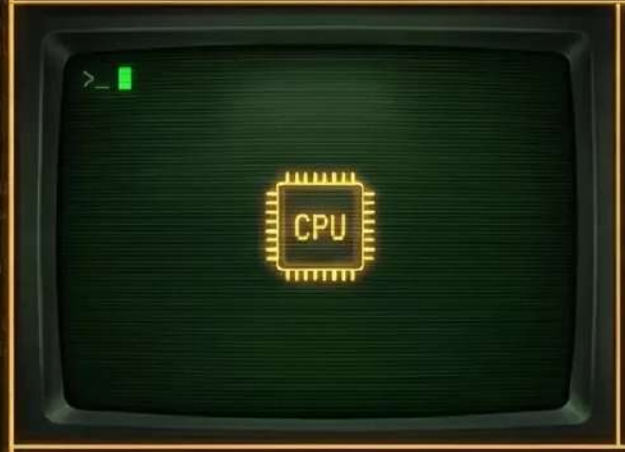
2. THE HEX EDITOR (DEBUG)

3. THE PATCHED GAME

FOR EDUCATIONAL PURPOSES ONLY. RESPECT COPYRIGHTS.

REAL vs. PROTECTED MODE

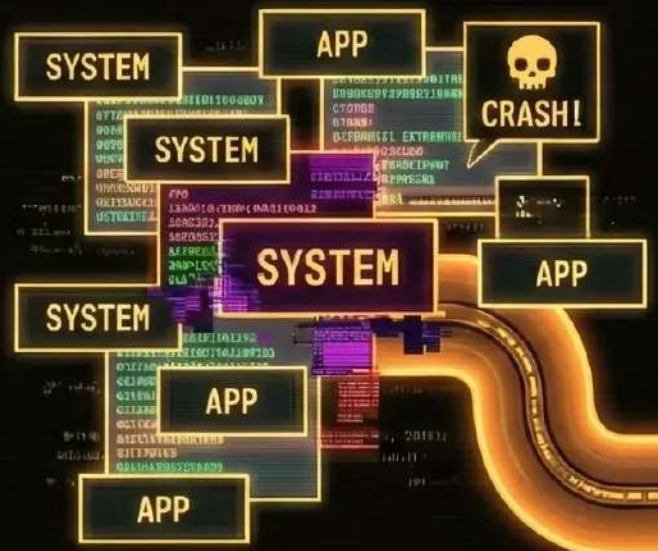
REAL MODE (16-BIT)



1MB RAM LIMIT
(SEGMENT:OFFSET)

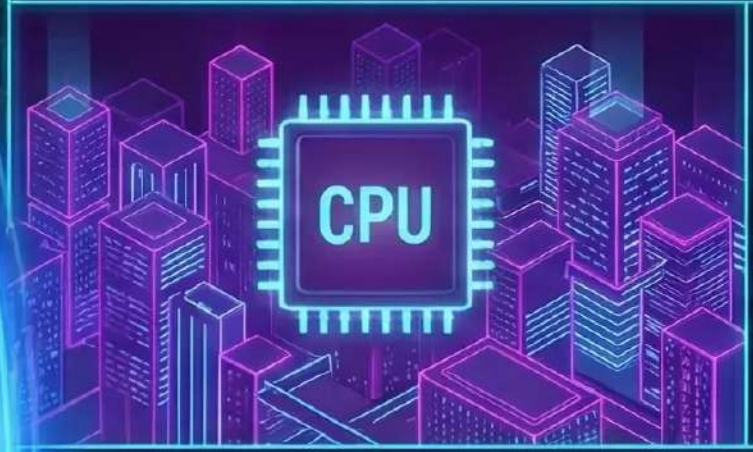
NO MEMORY PROTECTION
(WILD WEST)

SINGLE TASKING



MODE SWITCH
GATEWAY

PROTECTED MODE (16/32-BIT+)



VIRTUAL MEMORY
(PAGING)

HARDWARE RINGS
(0-3 SECURITY)

MULTITASKING
ENABLED

>_ ACCESSING NEXT LEVEL ARCHITECTURE...

16-Bit x86 Register Set

Reg	Name	Primary Function in DEBUG context
AX	Accumulator	Primary arithmetic and logic; Input/Output operations; Return values.
BX	Base	Base pointer for memory access; High-order word of file size.
CX	Count	Loop counters; Low-order word of file size.
DX	Data	I/O port addressing; High-order word for multiplication/division.
SP	Stack Pointer	Pointer to the top of the stack (grows downwards).
BP	Base Pointer	Stack frame base pointer for accessing local variables.
SI	Source Index	Source pointer for string operations.
DI	Dest Index	Destination pointer for string operations.
CS	Code Segment	Segment containing the currently executing instructions.
DS	Data Segment	Default segment for data variables.
SS	Stack Segment	Segment containing the stack.
ES	Extra Segment	Auxiliary segment; Critical for INT 13h disk buffer pointers.
IP	Instruction Ptr	Offset of the next instruction to be executed.
FL	Flags	Status indicators (Zero, Carry, Overflow, Sign, etc.).

DOS Debug

- Debug "hello world"
- Flipfont

```
1 mov ah, 9
2 mov dx, 108
3 int 21
4 ret
5 'Hello world', D, A, '$'
```

```
1 E 100 FA BA C4 03 B8 02 04 EF B8 04 06 EF BA CE 03 B8
2 E 110 04 02 EF B8 05 00 EF B8 06 00 EF B8 00 A0 8E C0
3 E 120 31 FF B9 00 01 51 B9 08 00 89 FE 8D 5D 0F 26 8A
4 E 130 04 26 86 07 26 88 04 46 4B E2 F3 83 C7 20 59 E2
5 E 140 E4 BA C4 03 B8 02 03 EF B8 04 02 EF BA CE 03 B8
6 E 150 04 00 EF B8 05 10 EF B8 06 0E EF FB B8 00 4C CD
7 E 160 21
```

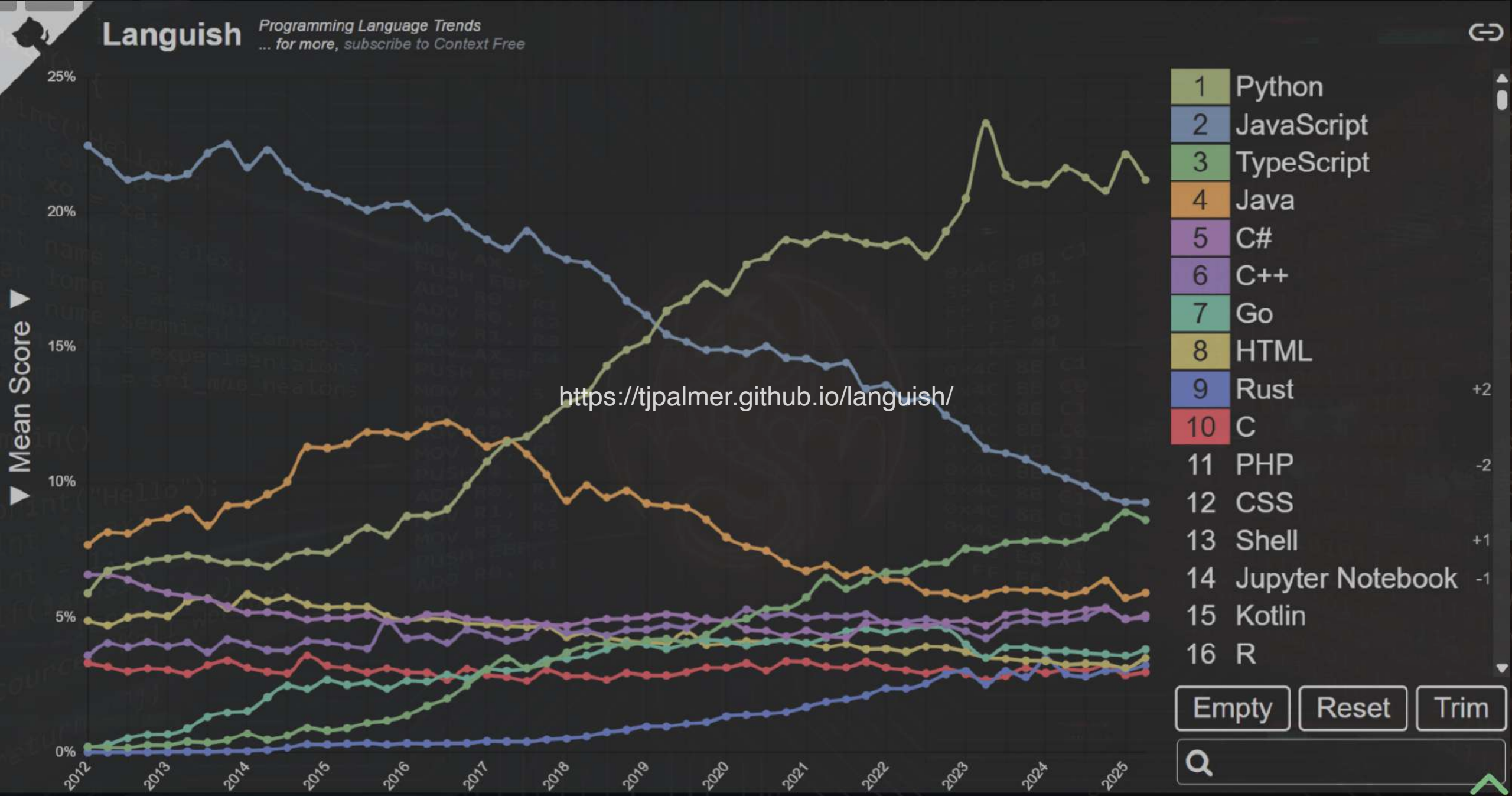
FreeDOS

DOSBox

```
C:\> debug
-?
assemble      A [address]
compare       C range address
dump          D [range]
enter         E address [list]
fill          F range list
go            G [=address] [addresses]
hex           H value1 value2
input         I port
load          L [address] [drive] [firstsector] [number]
move          M range address
name          N [pathname] [arglist]
output        O port byte
proceed       P [=address] [number]
quit          Q
register      R [register]
search        S range list
trace         T [=address] [value]
unassemble    U [range]
write         W [address] [drive] [firstsector] [number]
allocate expanded memory         XA [#pages]
deallocate expanded memory       XD [handle]
map expanded memory pages        XM [Lpage] [Ppage] [handle]
display expanded memory status   XS
```

Most Popular Languages

- **Sources**:
  - **TIOBE**: tiobe.com/tiobe-index
  - **Stack Overflow** 2025 Survey: survey.stackoverflow.co
  - **IEEE**: spectrum.ieee.org/top-programming-languages-2025
  - **GitHub** Octoverse 2025: github.blog/news-insights/octoverse/
  - **PYPL**: pypl.github.io I github.com/pypl
  - **Languish**: tjpalmer.github.io/languish I github.com/tjpalmer/languish
  - RedMonk: redmonk.com/data/



Data from GitHub and Stack Overflow

PYPL PopularitY of Programming Languages

Rank	Change	Language	Share	1-year trend
1		Python	24.61 %	-5.0 %
2	▲▲	C/C++	14.13 %	+6.9 %
3	▲▲▲▲▲▲	Objective-C	13.35 %	+10.5 %
4	▼▼	Java	10.45 %	-4.8 %
5	▲	R	6.15 %	+1.6 %
6	▼▼▼	JavaScript	4.68 %	-3.3 %
7	▲▲▲▲	Swift	3.68 %	+1.1 %
8	▼	PHP	2.95 %	-0.9 %
9	▼▼▼▼	C#	2.79 %	-3.4 %
10	▲▲▲▲▲	Ada	2.71 %	+1.5 %

https://delphi.org/rankings/PYPL.html

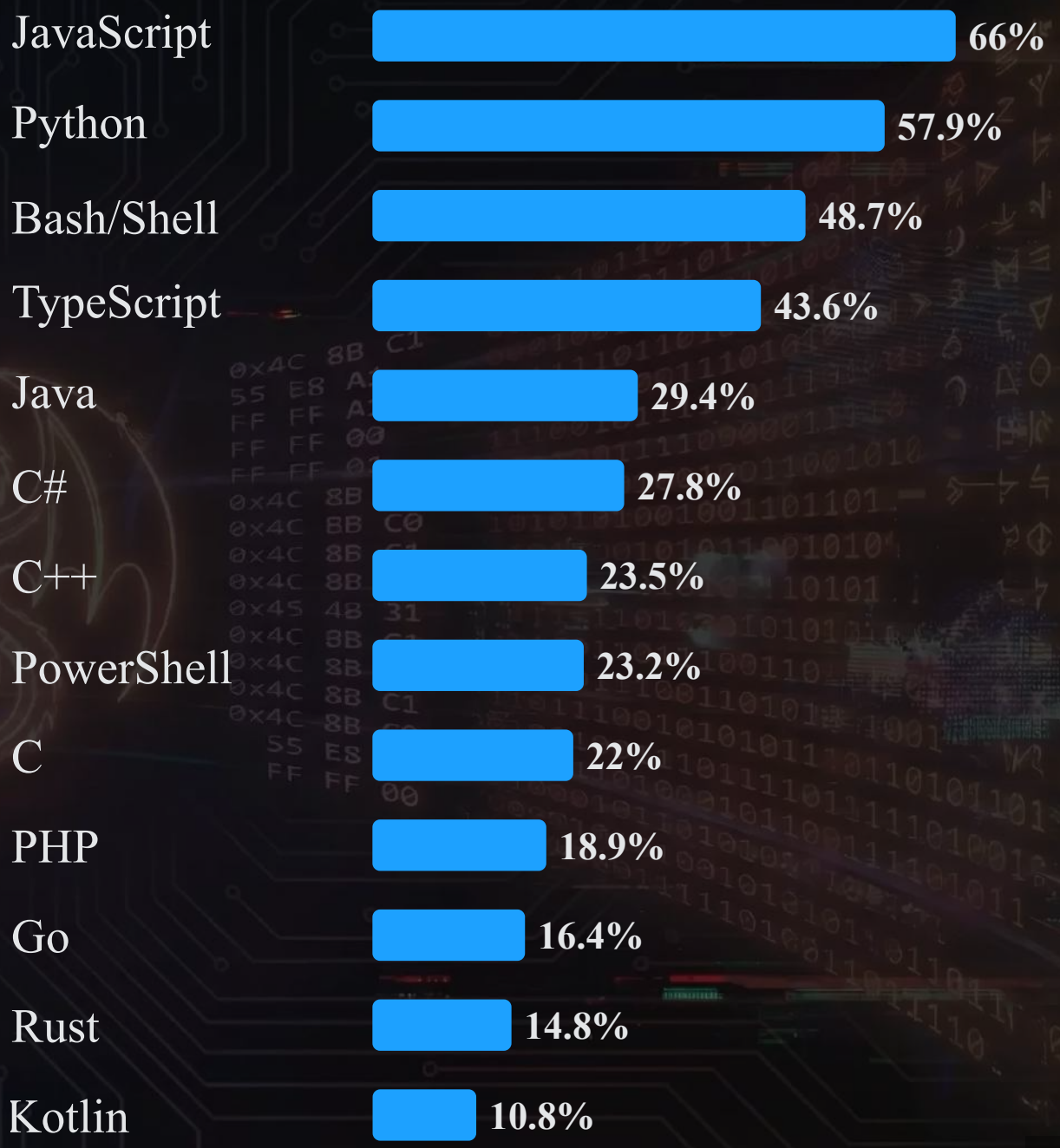
January 2026 © Pierre Carbonnelle pypl.github.io

pypl.github.io I github.com/pypl

Determined by analyzing frequency of language tutorials are searched on Google

Stackoverflow

- 2025 Survey Results
- For *"All Respondents"*
- Responses: 31,771 (64.8%)
- (removed HTML & SQL)



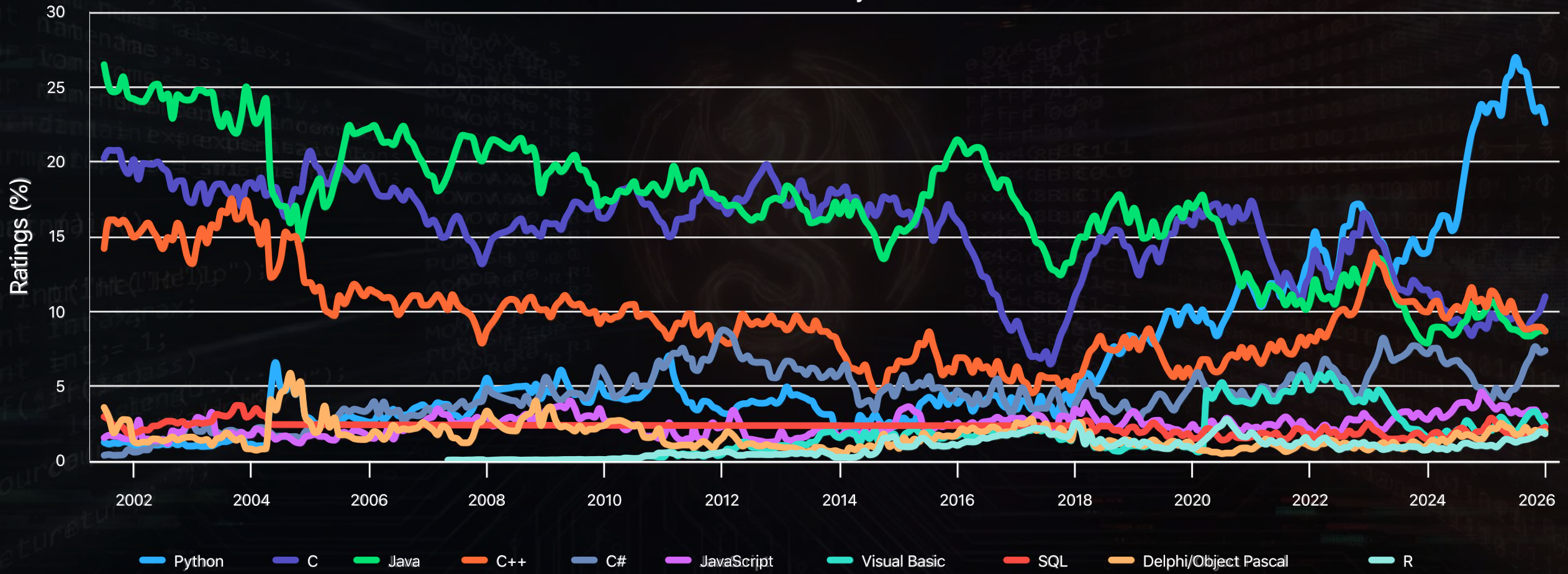
TIOBE

- Ranking in January 2026
- Based on search data

Jan 2026	Jan 2025	Change	Programming Language	Ratings	Change
1	1		Python	22.61%	-0.68%
2	4	▲	C	10.99%	0.0213
3	3		Java	8.71%	-1.44%
4	2	▼	C++	8.67%	-1.62%
5	5		C#	7.39%	0.0294
6	6		JavaScript	3.03%	-1.17%
7	9		Visual Basic	2.41%	0.0004
8	8		SQL	2.27%	-0.14%
9	11	▲	Delphi/Object Pascal	1.98%	0.0019
10	18	▲▲	R	1.82%	0.0081
11	32	▲▲	Perl	1.63%	0.0114
12	10	▼	Fortran	1.61%	-0.42%
13	14	▲	Rust	1.51%	0.0034
14	15	▲	MATLAB	1.40%	0.0034
15	13	▼	PHP	1.38%	0.00%
16	7	▼▼	Go	1.24%	-1.37%
17	12	▼▼	Scratch	1.24%	-0.31%
18	26	▲▲	Ada	1.19%	0.0054
19	17	▼	Assembly language	1.07%	0.0005
20	25	▲▲	Kotlin	0.97%	0.0023

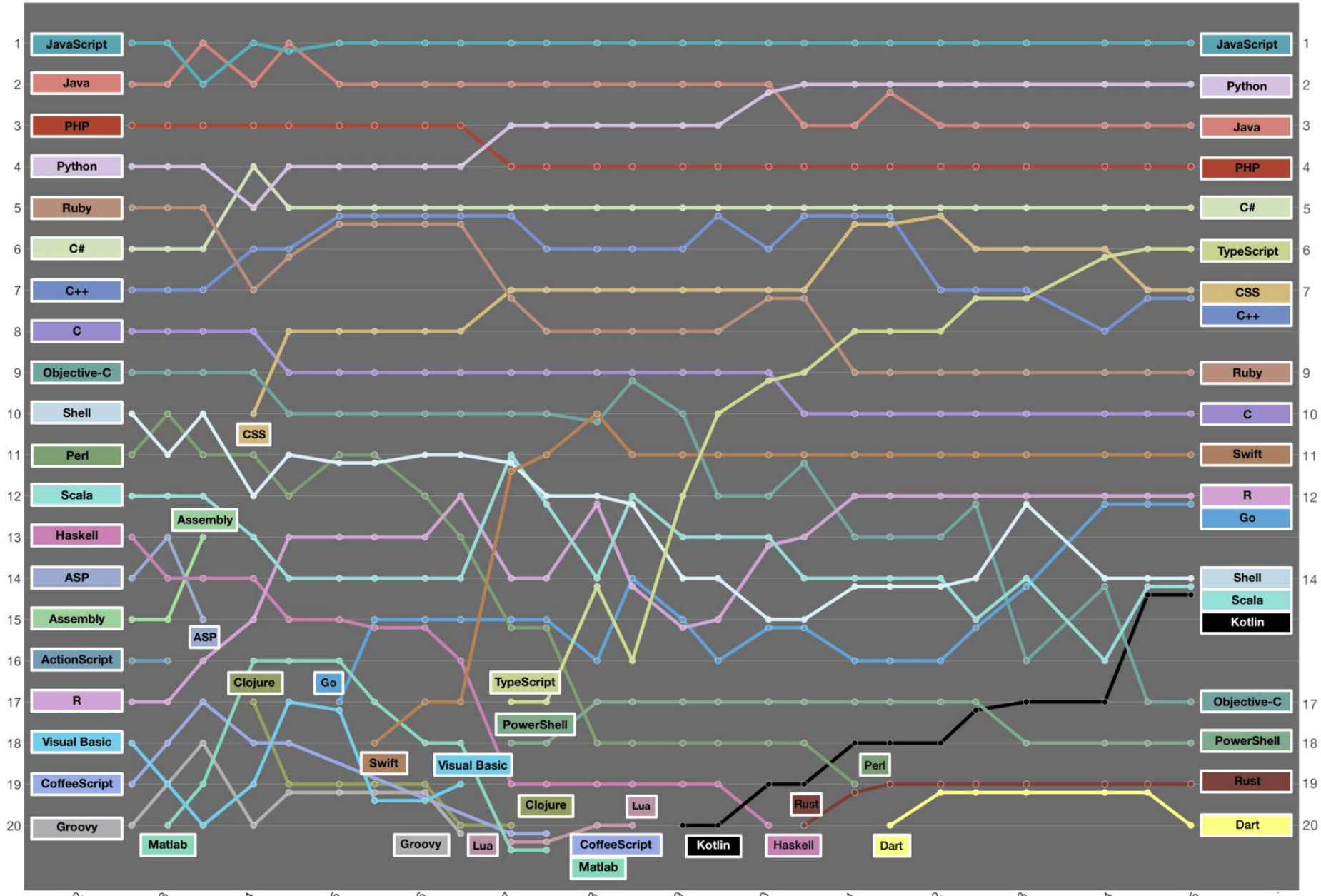
TIOBE

January 2026



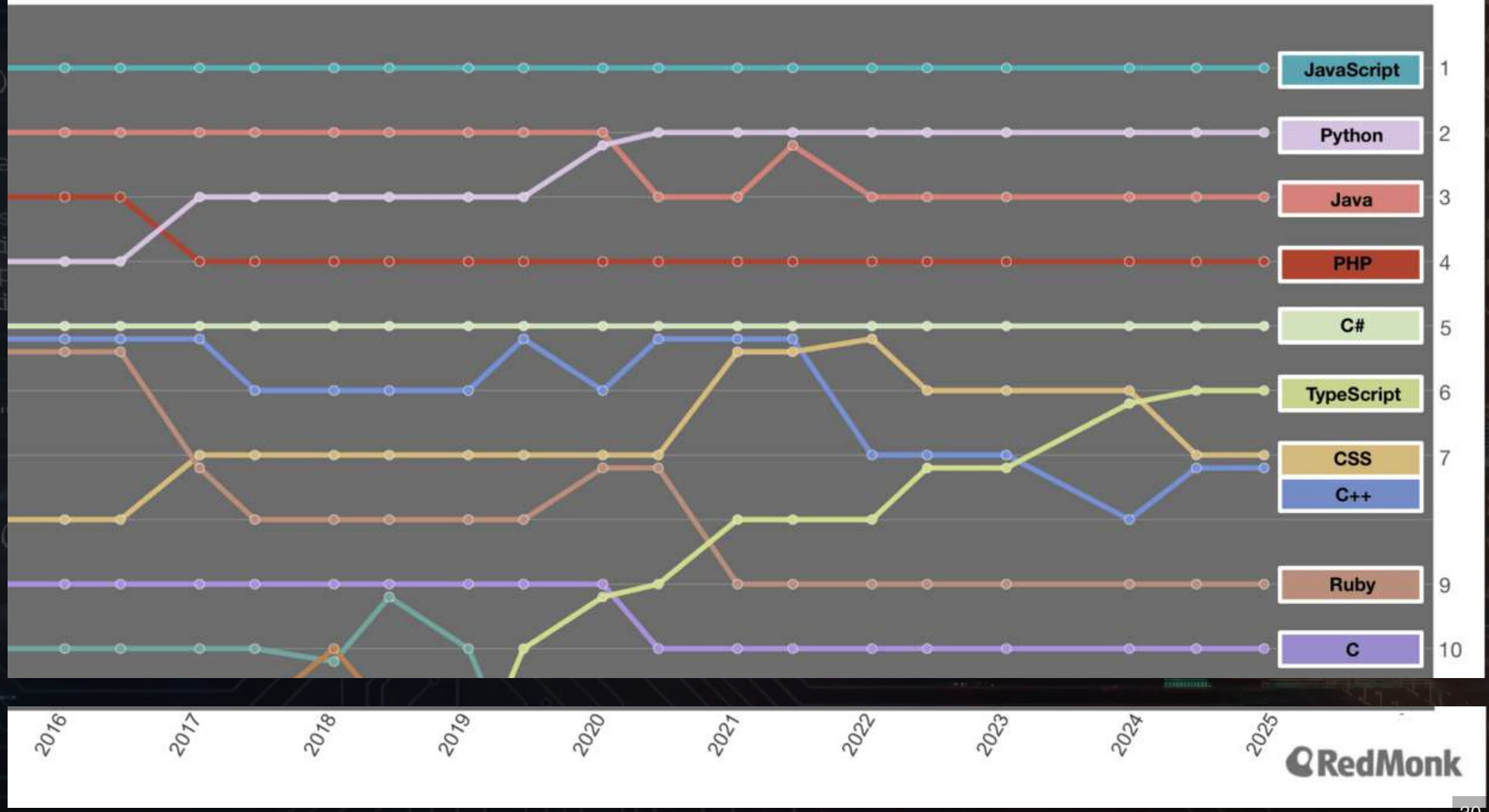
RedMonk Language Rankings

September 2012 - December 2024



RedMonk Language Rankings

September 2012 - December 2024



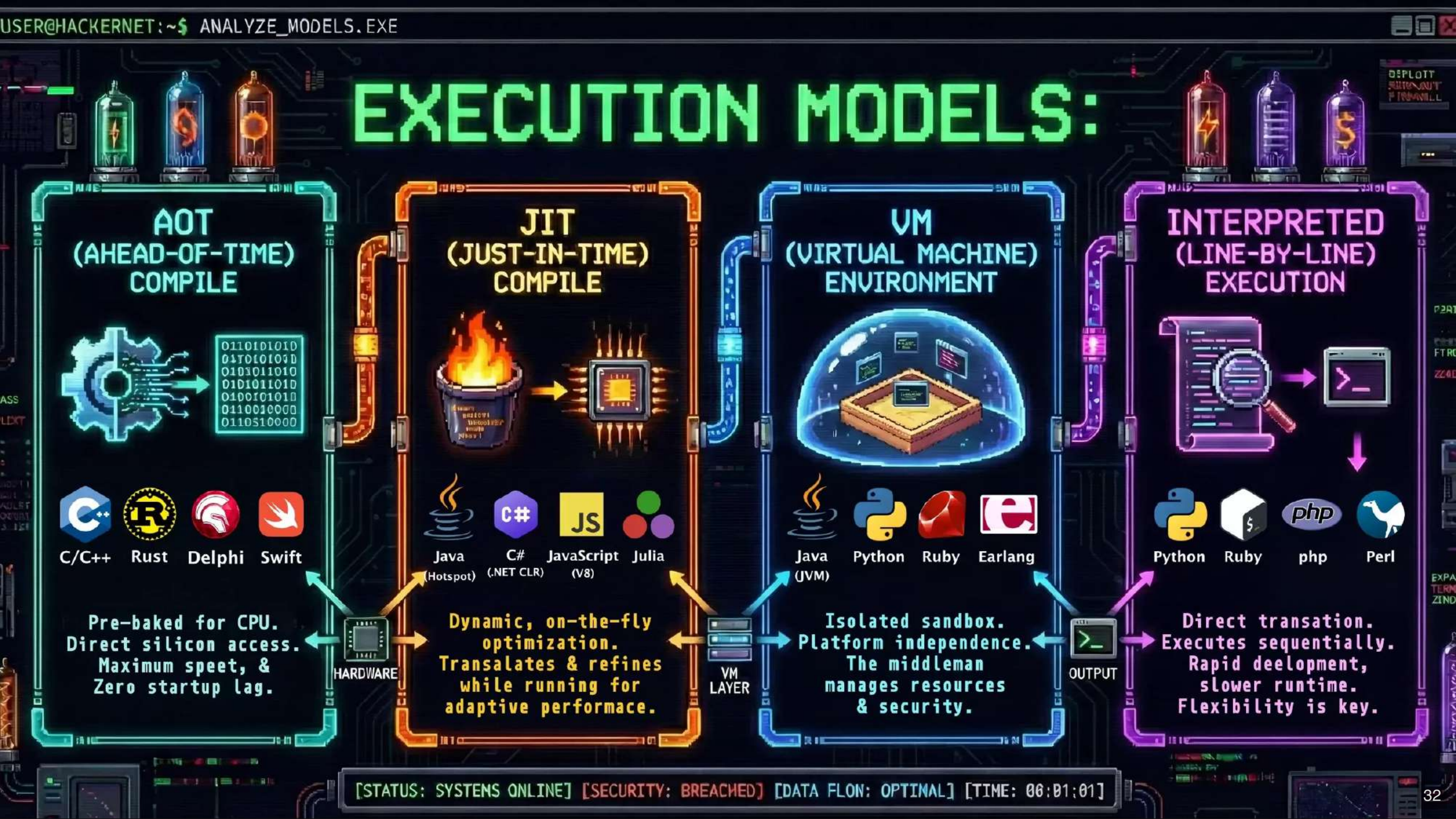
Overall

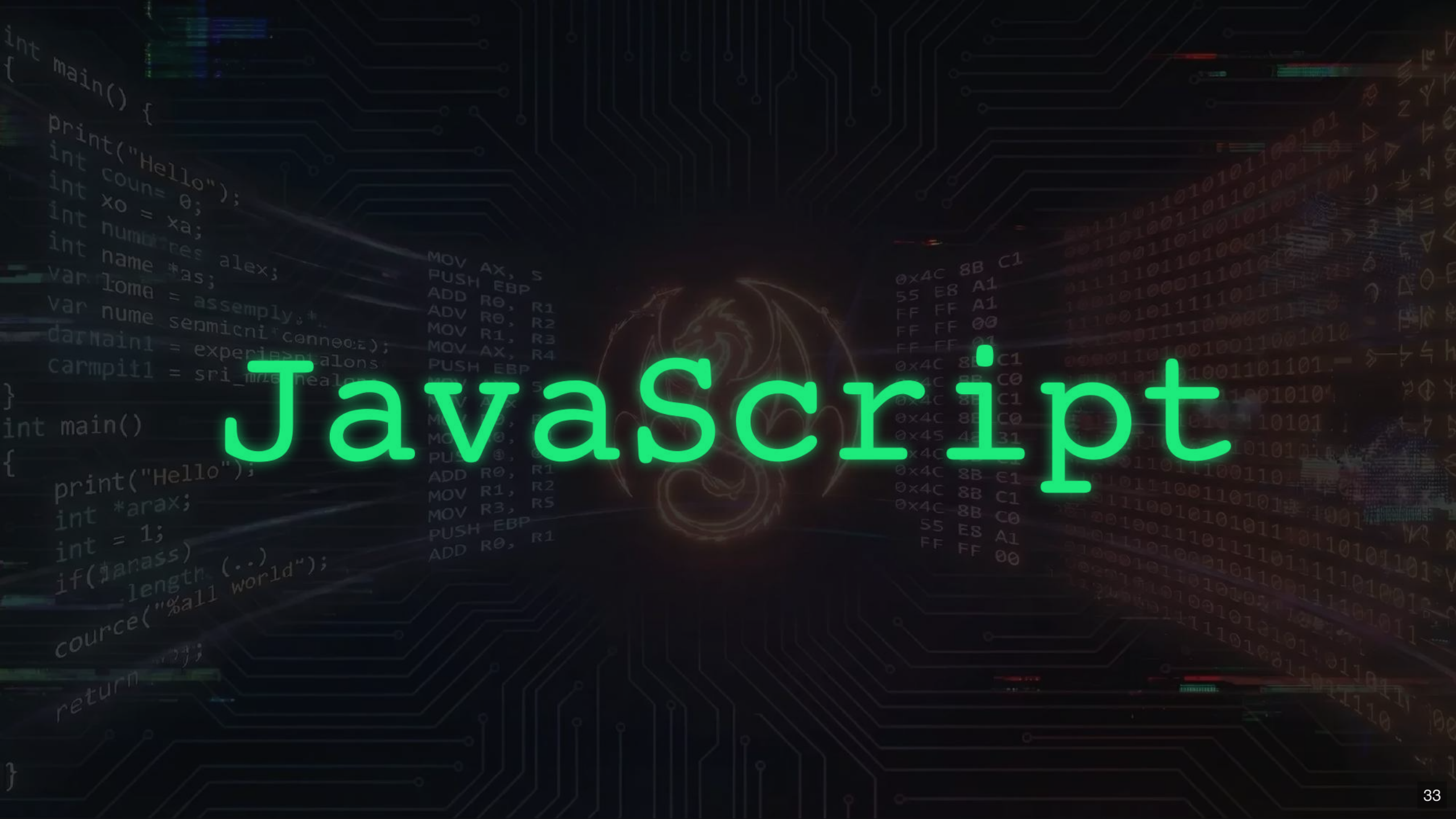
Top 10*ish*

- JavaScript/TypeScript
- Python
- Java
- C#
- PHP
- C/C++
- Ruby
- Bash/Shell/PowerShell

Other Common

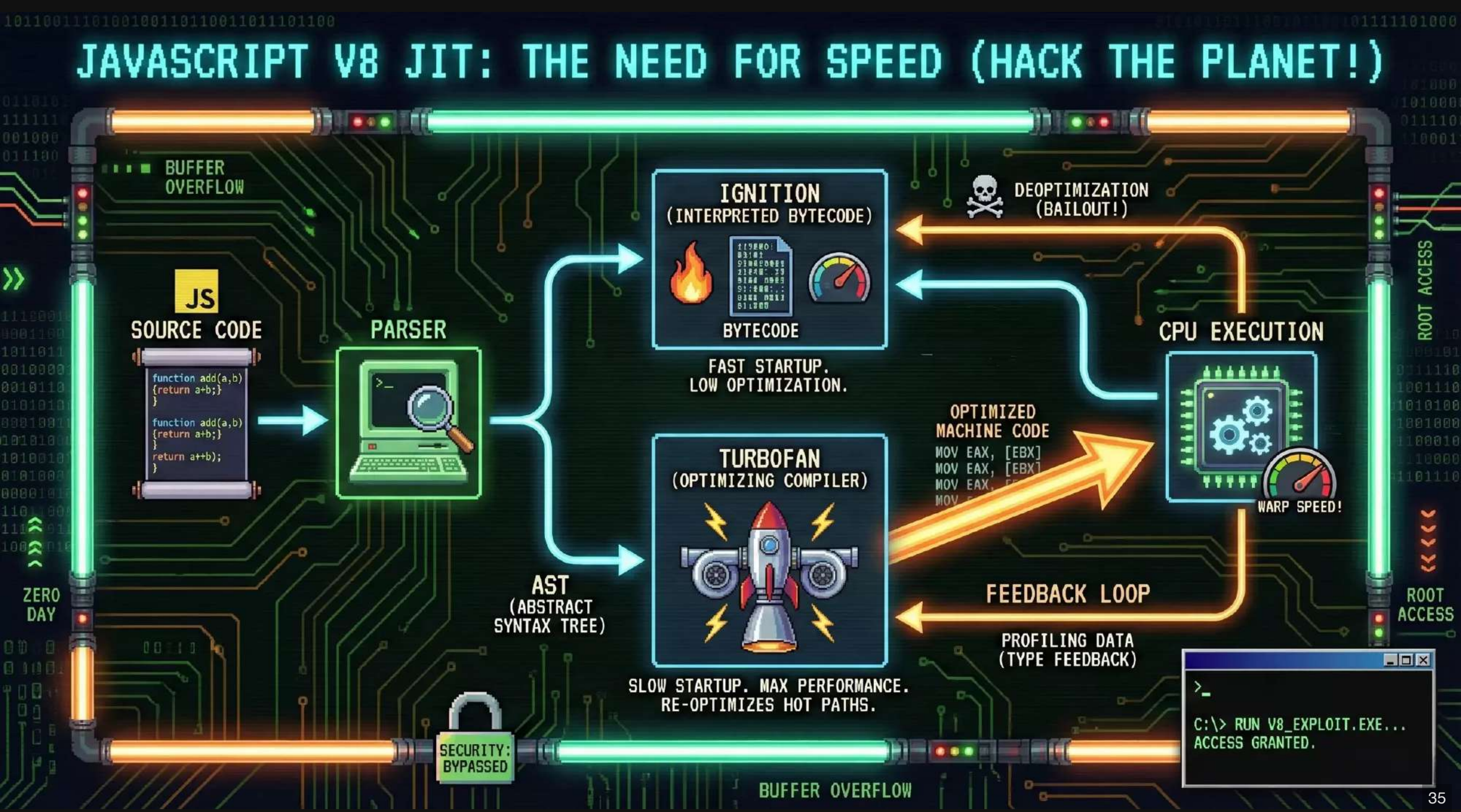
- Ada
- Rust
- Go
- Kotlin
- Swift
- R
- Objective-C
- Visual Basic
- Delphi/Pascal


USER@HACKERNET:~$ ANALYZE_MODELS.EXE
EXECUTION MODELS:
AOT
(AHEAD-OF-TIME)
COMPILE
C/C++
Rust
Delphi
Swift
Pre-baked for CPU.
Direct silicon access.
Maximum speet, &
Zero startup lag.
HARDWARE
JIT
(JUST-IN-TIME)
COMPILE
Java
(Hotspot)
C#
(.NET CLR)
JavaScript
(V8)
Julia
Dynamic, on-the-fly
optimization.
Transalates & refines
while running for
adaptive performace.
VM
LAYER
VM
(VIRTUAL MACHINE)
ENVIRONMENT
Java
(JVM)
Python
Ruby
Earlang
Isolated sandbox.
Platform independence.
The middleman
manages resources
& security.
OUTPUT
INTERPRETED
(LINE-BY-LINE)
EXECUTION
Python
Ruby
php
Perl
Direct transation.
Executes sequentially.
Rapid deelopment,
slower runtime.
Flexibility is key.
[STATUS: SYSTEMS ONLINE] [SECURITY: BREACHED] [DATA FLON: OPTIMAL] [TIME: 00:01:01]


JavaScript

JavScript Debugging

```
1  console.log("The basic log");
2
3  // Group related logs to keep the console tidy
4  console.group("User Transaction");
5  console.log("User: Alice");
6  console.log("Item: Sword of Truth");
7  console.groupEnd();
8
9  // Display arrays of objects as a clean table
10 const users = [
11   { name: "Alice", role: "Mage", hp: 100 },
12   { name: "Bob", role: "Warrior", hp: 150 }
13 ];
14 console.table(users);
15
16 // Only logs if the condition is false
17 console.assert(users[0].hp > 0, "User is dead!", users[0]);
18
19 // Print a stack trace
20 console.trace();
```


JAVASCRIPT V8 JIT: THE NEED FOR SPEED (HACK THE PLANET!)
BUFFER OVERFLOW
JS
SOURCE CODE
function add(a,b) {return a+b;}
function add(a,b) {return a+b;}
return a++b);
PARSER
AST
(ABSTRACT SYNTAX TREE)
IGNITION
(INTERPRETED BYTECODE)
BYTECODE
FAST STARTUP.
LOW OPTIMIZATION.
TURBOFAN
(OPTIMIZING COMPILER)
SLOW STARTUP. MAX PERFORMANCE.
RE-OPTIMIZES HOT PATHS.
DEOPTIMIZATION
(BAILOUT!)
OPTIMIZED
MACHINE CODE
MOV EAX, [EBX]
MOV EAX, [EBX]
CPU EXECUTION
WARP SPEED!
FEEDBACK LOOP
PROFILING DATA
(TYPE FEEDBACK)
ROOT ACCESS
ZERO DAY
ROOT ACCESS
SECURITY: BYPASSED
BUFFER OVERFLOW
C:\> RUN V8_EXPLOIT.EXE...
ACCESS GRANTED.

JavScript Disassembly

```
1  function square(a) {
2      let result = a * a;
3      return result;
4  }
5
6  // Collect type information on next call of function
7  %PrepareFunctionForOptimization(square)
8
9  // Call function once to fill type information
10 square(23);
11
12 // Call function again to go from uninitialized
13 //              -> pre-monomorphic -> monomorphic
14 square(13);
15 %OptimizeFunctionOnNextCall(square);
16 square(71);
```

godbolt.org/z/MhKG9vvej

JavScript Disassembly

godbolt.org/z/MhKG9vvej

```
--- Raw source ---
(a) {
        let result = a * a;
        return result;
}

--- Optimized code ---
optimization_id = 0
source_position = 15
kind = TURBOFAN
name = square
stack_slots = 6
compiler = turbofan
address = 0x3b0800002259

Instructions (size = 284)
0x5d99e0004040     0  8b59f4            movl rbx,[rcx-0xc]
0x5d99e0004043     3  4d8b95d0010000    REX.W movq r10,[r13+0x1d0]
0x5d99e000404a     a  490bda            REX.W orq rbx,r10
0x5d99e000404d     d  f6431a20          testb [rbx+0x1a],0x20
0x5d99e0004051    11  0f85293ddea0      jnz 0x5d9980de7d80  (CompileLazyDeoptimizedCode)    ;; near builtin entry
0x5d99e0004057    17  55                push rbp
0x5d99e0004058    18  4889e5            REX.W movq rbp,rsp
0x5d99e000405b    1b  56                push rsi
0x5d99e000405c    1c  57                push rdi
0x5d99e000405d    1d  50                push rax
0x5d99e000405e    1e  4883ec08          REX.W subq rsp,0x8
0x5d99e0004062    22  488975e0          REX.W movq [rbp-0x20],rsi
0x5d99e0004066    26  493b65a0          REX.W cmpq rsp,[r13-0x60] (external value (StackGuard::address_of_jslimit()))
```

```
Inlined functions (count = 0)

Deoptimization Input Data (deopt points = 4)
 index  bytecode-offset    pc
     0                2    NA
     1                2    NA
     2                2    NA
     3               -1    b7

Safepoints (entries = 2, byte size = 20)
0x5d99e00040f7     b7  slots (sp->fp): 10000000  deopt      3 trampoline:    118
0x5d99e0004122     e2  slots (sp->fp): 00000000

RelocInfo (size = 9)
0x5d99e0004053  near builtin entry
0x5d99e00040ea  full embedded object  (0x10a400183c85 <NativeContext[285]>)
0x5d99e00040f3  near builtin entry
0x5d99e000411e  near builtin entry
```

vs.

GCC vs LLVM

Compiler Infrastructure Platforms

GCC vs LLVM

	GCC	**LLVM**
Initial release	March 22, 1987; 38 years ago	2003; 23 years ago
Original authors	Richard Stallman	Chris Lattner, Vikram Adve
Stable release	15.2 / 8 Aug 2025	21.1.8 / 16 Dec 2025
Written in	C, C++	C++
Languages	49	27
OS	Cross-platform	Cross-platform
Size	~15 million LOC	~35.56 million LOC
License	GPLv3+ with GCC Runtime Library Exception	Apache License 2.0 with LLVM Exceptions

GCC vs LLVM

Feature	GCC	LLVM
Primary License	GPLv3 (Copyleft). Ensures derivative works remain open-source.	Apache 2.0 with LLVM Exceptions (Permissive). Allows for proprietary commercial extensions.
Architecture	Monolithic. Older design with tightly coupled components. Difficult to use its internal parts as separate libraries.	Modular. Designed as a set of reusable libraries with clear interfaces. Easy to integrate into other tools.
Intermediate Representation (IR)	Uses GIMPLE and RTL internally. These are not designed for external use or tooling.	Uses LLVM IR, a universal, language-independent, assembly-like language designed for analysis and optimization by external tools.
Primary Frontends	gcc (C), g++ (C++), gfortran (Fortran), Ada, Go, D, Objective-C.	Clang is the primary frontend for C, C++, and Objective-C. Used as a backend by Rust, Swift, Julia, Ruby, etc.
Platform Support	Extremely broad, including many legacy and niche architectures. The standard for Linux and embedded systems.	Broad support for modern architectures (x86, ARM, RISC-V, Wasm). The default on macOS, iOS, and widely used in Android NDK.
Compilation Speed	generally slower, especially at higher optimization levels.	Generally faster, particularly with the Clang frontend. Known for fast, incremental builds.
Generated Code Performance	Mature and highly optimized. Historically had a slight edge in some CPU-intensive benchmarks.	Highly competitive. Often matches or beats GCC, especially with modern features like advanced vectorization and Link-Time Optimization (LTO).
Diagnostics & Tooling	Traditional error messages. Tooling includes GDB and GNU binutils.	Superior, human-readable error messages. Rich ecosystem of modular tools like LLDB, Clang Static Analyzer, clang-tidy, and clang-format.
Key Strengths	Maturity, stability, incredibly wide hardware support, and strict adherence to open-source principles.	Modularity, build speed, excellent diagnostics, modern design, and permissiveness for commercial use.

GCC, the GNU Compiler Collection

The GNU Compiler Collection includes front ends for C, C++, Objective-C, Objective-C++, Fortran, Ada, Go, D, Modula-2, COBOL, Rust, and Algol 68 as well as libraries for these languages (libstdc++,...). GCC was originally written as the compiler for the GNU operating system. The GNU system was developed to be 100% free software, free in the sense that it respects the user's freedom.

We strive to provide regular, high quality releases, which we want to work well on a variety of native and cross targets (including GNU/Linux), and encourage everyone to contribute changes or help testing GCC. Our sources are readily and freely available via Git and weekly snapshots.

Major decisions about GCC are made by the steering committee, guided by the mission statement.

News

GCC developer room at FOSDEM 2026: Schedule Available [2025-12-15]
FOSDEM 2026: Brussels, Belgium, Jan 31 - Feb 1, 2026

Algol 68 front end added [2025-11-30]
An experimental Algol 68 programming language front end has been added to GCC.

GCC developer room at FOSDEM 2026: Call for Participation [2025-11-03]
FOSDEM 2026: Brussels, Belgium, Jan 31 - Feb 1, 2026

GCC 15.2 released [2025-08-08]

GNU Tools Cauldron 2025 [2025-08-01]
Porto, Portugal, September 26-28 2025

GCC 12.5 released [2025-07-11]

GCC 13.4 released [2025-06-05]

GCC 14.3 released [2025-05-23]

GCC 15.1 released [2025-04-25]

COBOL front end added [2025-04-17]
The COBOL programming language front end has been added to GCC. This front end was contributed by COBOLworx.

Older news | More news? Let gerald@pfeifer.com know!

Supported Releases

GCC 15.2 (changes)
Status: 2025-08-08 (regression fixes & docs only).
Serious regressions. All regressions.

GCC 14.3 (changes)
Status: 2025-05-23 (regression fixes & docs only).
Serious regressions. All regressions.

GCC 13.4 (changes)
Status: 2025-06-05 (regression fixes & docs only).
Serious regressions. All regressions.

Development: GCC 16.0 (release criteria, changes)
Status: 2026-01-12 (regression fixes & docs only).
Serious regressions. All regressions.

Search our site

Search

Match: All words Sort by: Newest

There is also a detailed search form.

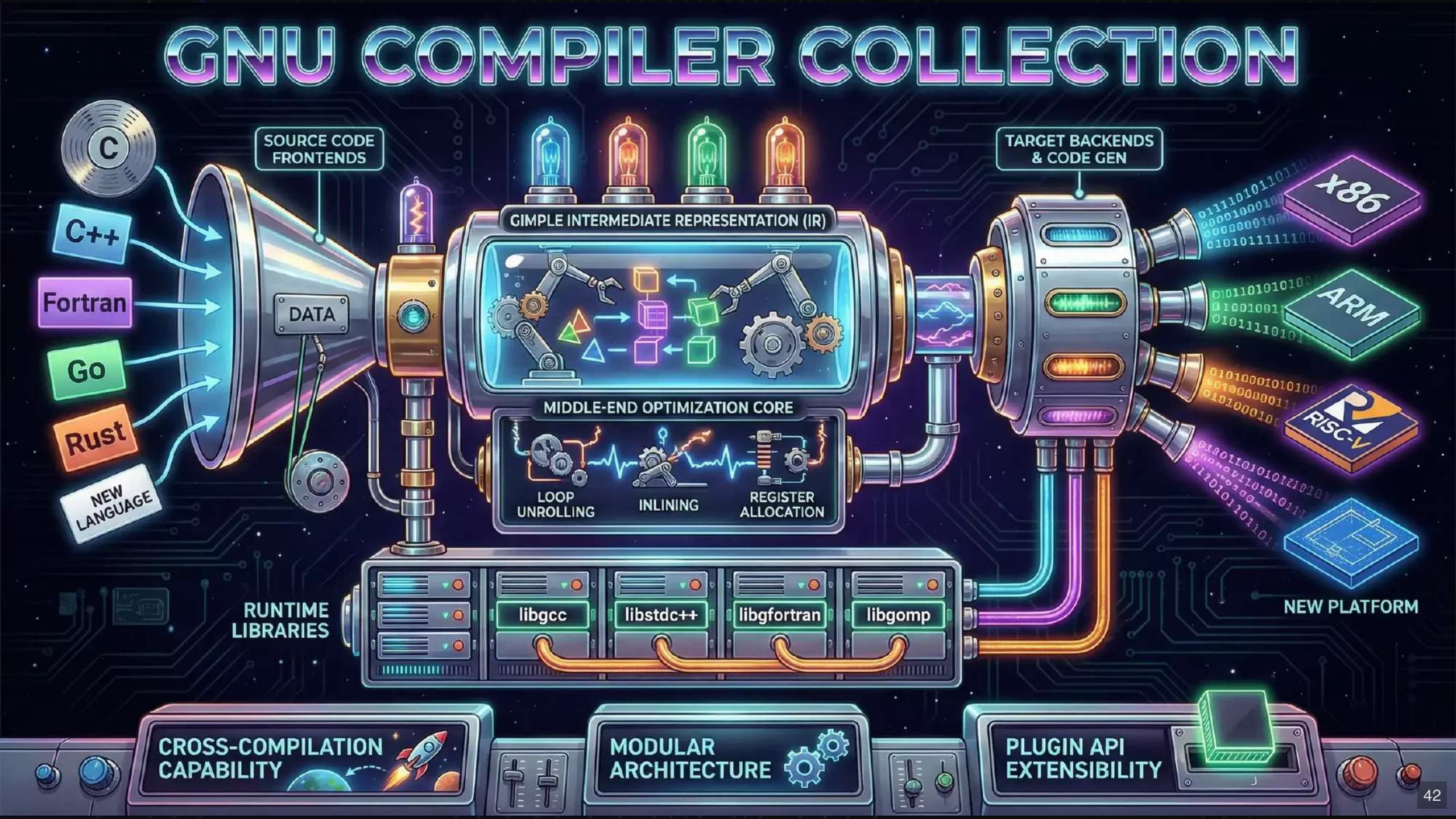
Get our announcements

your e-mail address Subscribe

FKA "GNU C Compiler"

https://gcc.gnu.org/

Initial release	March 22, 1987; 38 years ago
Original author	Richard Stallman
Stable release	15.2 / 8 Aug 2025
Written in	C, C++
Languages	49
OS	Cross-platform
Size	~15 million LOC
License	GPLv3+ with GCC Runtime Library Exception


GNU COMPILER COLLECTION
C
C++
Fortran
Go
Rust
NEW LANGUAGE
SOURCE CODE FRONTENDS
DATA
GIMPLE INTERMEDIATE REPRESENTATION (IR)
MIDDLE-END OPTIMIZATION CORE
LOOP UNROLLING
INLINING
REGISTER ALLOCATION
TARGET BACKENDS & CODE GEN
x86
ARM
RISC-V
NEW PLATFORM
RUNTIME LIBRARIES
libgcc
libstdc++
libgfortran
libgomp
CROSS-COMPILATION CAPABILITY
MODULAR ARCHITECTURE
PLUGIN API EXTENSIBILITY

GCC COMPILER OPTIMIZATIONS

MAXIMIZING CODE PERFORMANCE THROUGH ADVANCED TRANSFORMATIONS

LOOP UNROLLING

REDUCES LOOP OVERHEAD BY EXECUTING MULTIPLE ITERATIONS IN A SINGLE STEP.

BEFORE

```
for (i=0; i<100; i++) {
  ...
}
```

AFTER

```
for (i=0; i<100; i++) {
  ...
  i=0, 4, 8...
  i=1, 5, 9...
  i=2, 6, 10...
  i=3, 7, 11...
}
```

INCREASED INSTRUCTION LEVEL PARALLELISM, FASTER EXECUTION.

INLINING

ELIMINATES FUNCTION CALL OVERHEAD BY INSERTING THE FUNCTION'S CODE DIRECTLY.

BEFORE

```
main() {
  foo()
}
foo()
}
```

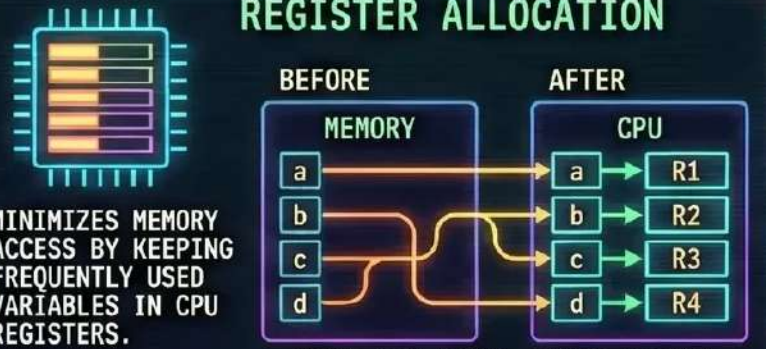
AFTER

```
main() {
  foo() {
    ...
  }
}
```

AVOIDS CONTEXT SWITCHING, IMPROVES CACHE LOCALITY.

REGISTER ALLOCATION

MINIMIZES MEMORY ACCESS BY KEEPING FREQUENTLY USED VARIABLES IN CPU REGISTERS.



FASTER DATA ACCESS, REDUCED LATENCY.

OTHER OPTIMIZATIONS

DEAD CODE ELIMINATION

REMOVES UNREACHABLE OR UNUSED CODE SECTIONS.

COMMON SUBEXPRESSION ELIMINATION

(a+b) (a+b)

AVOIDS REDUNDANT COMPUTATIONS.

VECTORIZATION

DATA x, y, z, w ....

EXPLOITS SIMD (SINGLE INSTRUCTION, MULTIPLE DATA) CAPABILITIES.

GCC: UNLEASHING PERFORMANCE THROUGH INTELLIGENT COMPILATION. OPTIMIZE RESPONSIBLY.

https://llvm.org/

The LLVM Compiler Infrastructure

Site Map:

- Overview
- Features
- Documentation
- Command Guide
- FAQ
- Publications
- LLVM Projects
- Open Projects
- LLVM Users
- Bug tracker
- LLVM Logo
- Blog
- Meetings
- LLVM Foundation

Download!

Download now:
LLVM 21.1.8
All Releases
APT Packages
RPM Snapshots
Pre-releases

View the open-source
license

Search this Site

Search!

LLVM Overview

The LLVM Project is a collection of modular and reusable compiler and toolchain technologies. Despite its name, LLVM has little to do with traditional virtual machines. The name "LLVM" itself is not an acronym; it is the full name of the project.

LLVM began as a research project at the University of Illinois, with the goal of providing a modern, SSA-based compilation strategy capable of supporting both static and dynamic compilation of arbitrary programming languages. Since then, LLVM has grown to be an umbrella project consisting of a number of subprojects, many of which are being used in production by a wide variety of commercial and open source projects as well as being widely used in academic research. Code in the LLVM project is licensed under the "Apache 2.0 License with LLVM exceptions"

The primary sub-projects of LLVM are:

1. The **LLVM Core** libraries provide a modern source- and target-independent optimizer, along with code generation support for many popular CPUs (as well as some less common ones!) These libraries are built around a well specified code representation known as the LLVM intermediate representation ("LLVM IR"). The LLVM Core libraries are well documented, and it is particularly easy to invent your own language (or port an existing compiler) to use LLVM as an optimizer and code generator.

2. **Clang** is an "LLVM native" C/C++/Objective-C compiler, which aims to deliver amazingly fast compiles, extremely useful error and warning messages and to provide a platform for building great source level tools. The Clang Static Analyzer and clang-tidy are tools that automatically find bugs in your code, and are great examples of the sort of tools that can be built using the Clang frontend as a library to parse C/C++ code.

3. The **LLDB** project builds on libraries provided by LLVM and Clang to provide a great native debugger. It uses the Clang ASTs and expression parser, LLVM JIT, LLVM disassembler, etc so that it provides an experience that "just works". It is also blazing fast and much more memory efficient than GDB at loading symbols.

4. The **libc++** and **libc++ ABI** projects provide a standard conformant and high-performance implementation of the C++ Standard Library, including full support for C++11 and C++14.

5. The **libc** project provides a high-performance, standards-conformant implementation of the C Standard Library.

Latest LLVM Release!

16 December 2025: LLVM 21.1.8 is now **available for download**! LLVM is publicly available under an open source License. Also, you might want to check out **the new features** in Git that will appear in the next LLVM release. If you want them early, download LLVM through anonymous Git.

Upcoming Events

2026 EuroLLVM Developers' Meeting

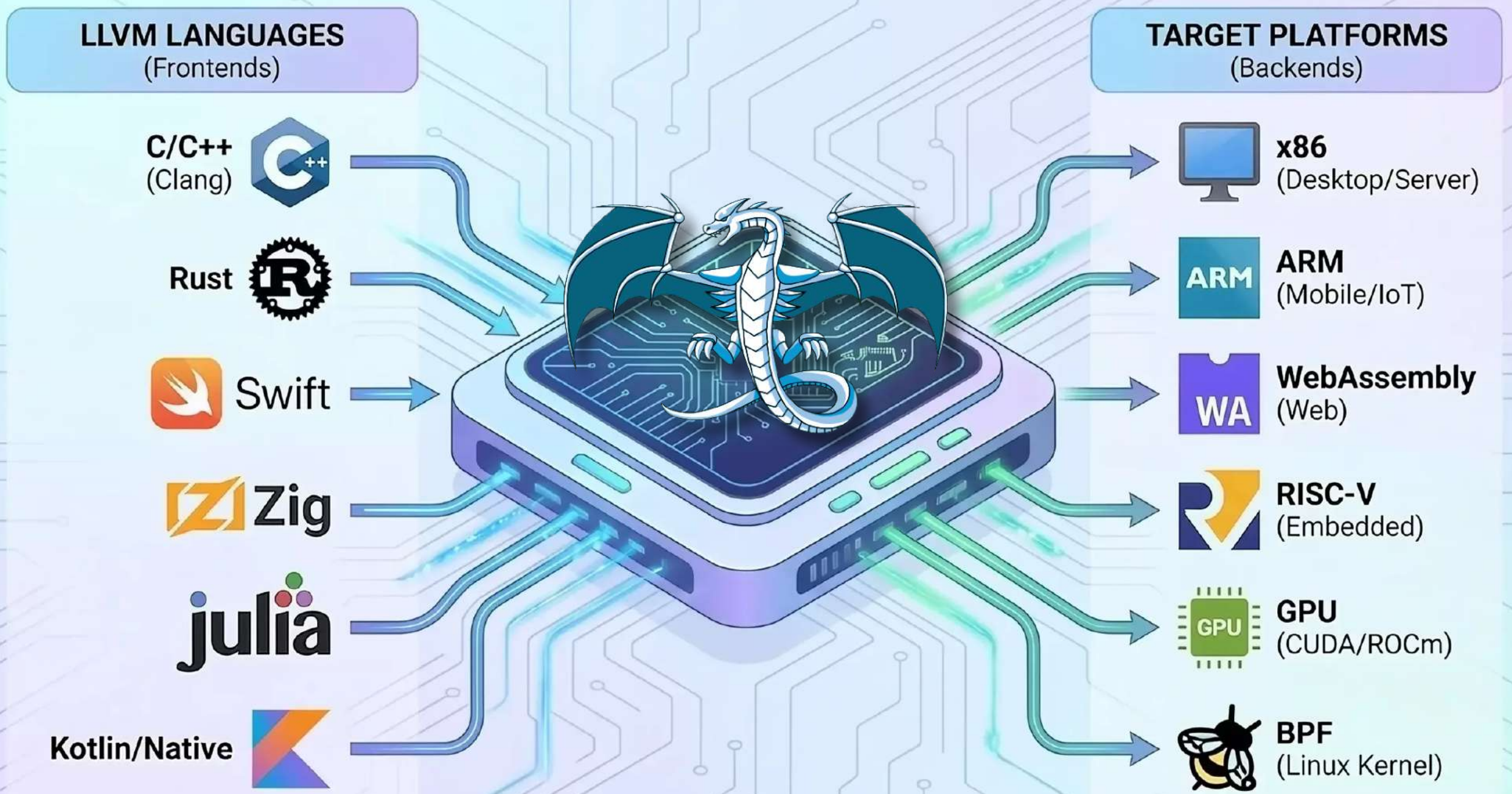
ACM Software System Award!

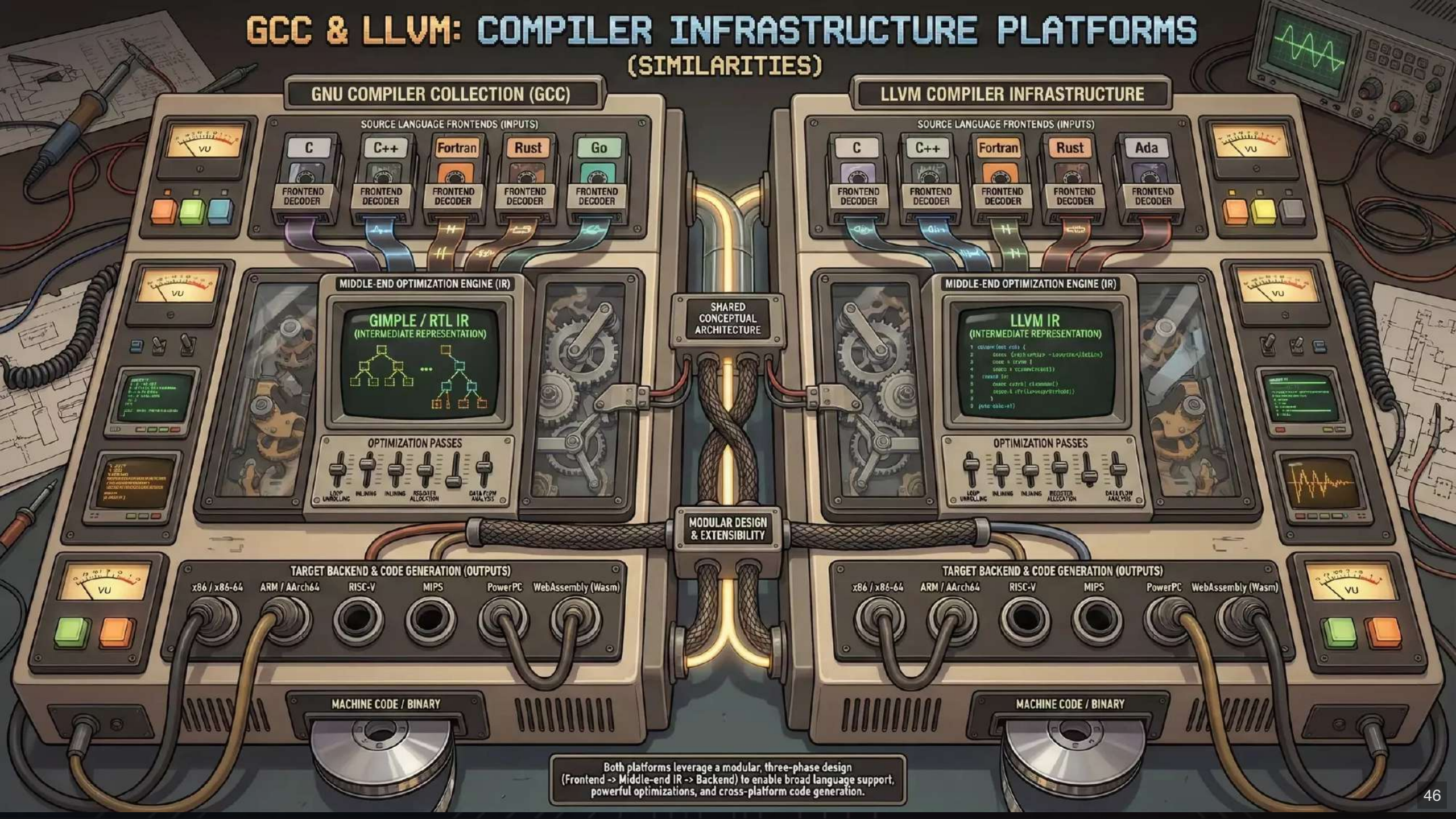
LLVM has been awarded the **2012 ACM Software System Award**! This award is given by ACM to *one* software system worldwide every year. LLVM is in highly distinguished company! Click on any of the individual recipients' names on that page for the detailed citation describing the award.

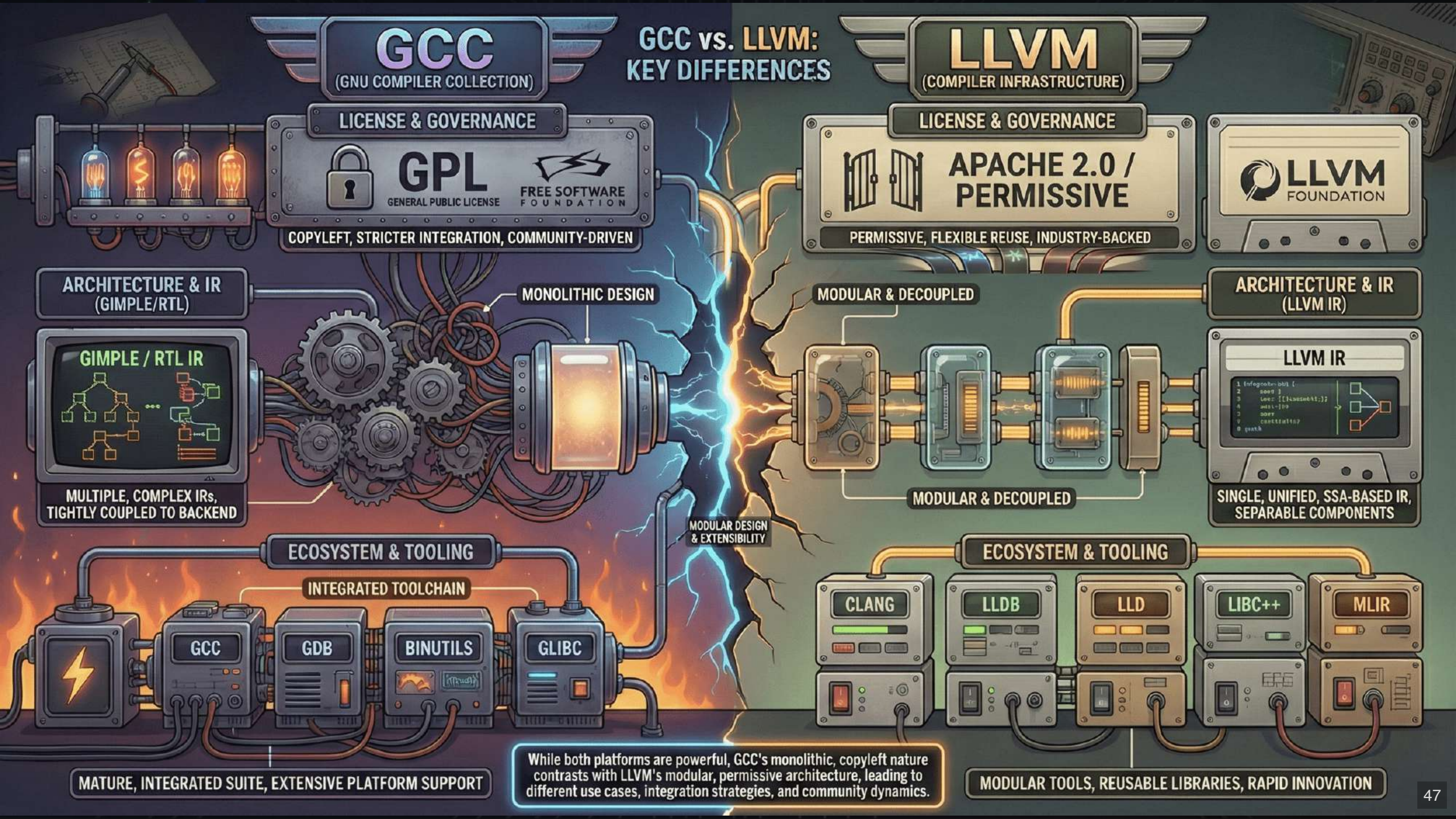
Upcoming Releases

LLVM Release Schedule:

- 22.1.x
  - Tue Jan 13th, 2026: release/22.x branch
  - Fri Jan 16th, 2026: 22.1.0-rc1
  - Tue Jan 27th, 2026: 22.1.0-rc2
  - Tue Feb 10th, 2026: 22.1.0-rc3
  - Tue Feb 24th, 2026: 22.1.0


LLVM LANGUAGES
(Frontends)
C/C++
(Clang)
Rust
Swift
Zig
julia
Kotlin/Native
TARGET PLATFORMS
(Backends)
x86
(Desktop/Server)
ARM
ARM
(Mobile/IoT)
WA
WebAssembly
(Web)
RISC-V
(Embedded)
GPU
GPU
(CUDA/ROCm)
BPF
(Linux Kernel)

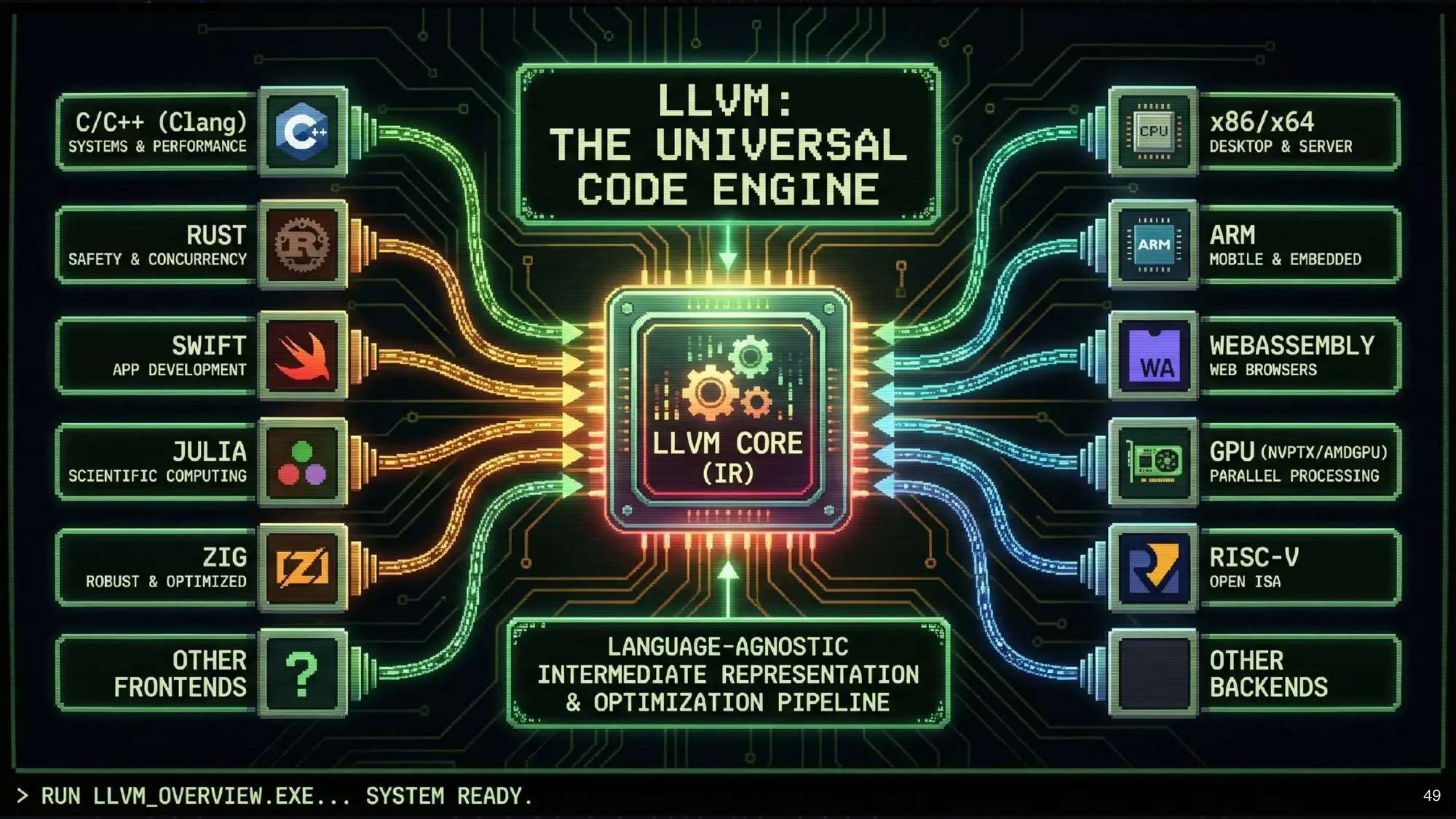

GCC & LLVM: COMPILER INFRASTRUCTURE PLATFORMS
(SIMILARITIES)
GNU COMPILER COLLECTION (GCC)
SOURCE LANGUAGE FRONTENDS (INPUTS)
C
C++
Fortran
Rust
Go
FRONTEND DECODER
MIDDLE-END OPTIMIZATION ENGINE (IR)
GIMPLE / RTL IR
(INTERMEDIATE REPRESENTATION)
OPTIMIZATION PASSES
LOOP UNROLLING
INLINING
INLINING
REGISTER ALLOCATION
DATA FLOW ANALYSIS
TARGET BACKEND & CODE GENERATION (OUTPUTS)
x86 / x86-64
ARM / AArch64
RISC-V
MIPS
PowerPC
WebAssembly (Wasm)
MACHINE CODE / BINARY
SHARED CONCEPTUAL ARCHITECTURE
MODULAR DESIGN & EXTENSIBILITY
LLVM COMPILER INFRASTRUCTURE
SOURCE LANGUAGE FRONTENDS (INPUTS)
C
C++
Fortran
Rust
Ada
FRONTEND DECODER
MIDDLE-END OPTIMIZATION ENGINE (IR)
LLVM IR
(INTERMEDIATE REPRESENTATION)
OPTIMIZATION PASSES
LOOP UNROLLING
INLINING
INLINING
REGISTER ALLOCATION
DATA FLOW ANALYSIS
TARGET BACKEND & CODE GENERATION (OUTPUTS)
x86 / x86-64
ARM / AArch64
RISC-V
MIPS
PowerPC
WebAssembly (Wasm)
MACHINE CODE / BINARY
Both platforms leverage a modular, three-phase design
(Frontend -> Middle-end IR -> Backend) to enable broad language support,
powerful optimizations, and cross-platform code generation.


GCC
(GNU COMPILER COLLECTION)
GCC vs. LLVM:
KEY DIFFERENCES
LLVM
(COMPILER INFRASTRUCTURE)
LICENSE & GOVERNANCE
GPL
GENERAL PUBLIC LICENSE
FREE SOFTWARE FOUNDATION
COPYLEFT, STRICTER INTEGRATION, COMMUNITY-DRIVEN
LICENSE & GOVERNANCE
APACHE 2.0 / PERMISSIVE
PERMISSIVE, FLEXIBLE REUSE, INDUSTRY-BACKED
LLVM FOUNDATION
ARCHITECTURE & IR
(GIMPLE/RTL)
MONOLITHIC DESIGN
GIMPLE / RTL IR
MULTIPLE, COMPLEX IRs,
TIGHTLY COUPLED TO BACKEND
MODULAR & DECOUPLED
ARCHITECTURE & IR
(LLVM IR)
LLVM IR
MODULAR & DECOUPLED
SINGLE, UNIFIED, SSA-BASED IR,
SEPARABLE COMPONENTS
MODULAR DESIGN
& EXTENSIBILITY
ECOSYSTEM & TOOLING
INTEGRATED TOOLCHAIN
GCC
GDB
BINUTILS
GLIBC
ECOSYSTEM & TOOLING
CLANG
LLDB
LLD
LIBC++
MLIR
MATURE, INTEGRATED SUITE, EXTENSIVE PLATFORM SUPPORT
While both platforms are powerful, GCC's monolithic, copyleft nature contrasts with LLVM's modular, permissive architecture, leading to different use cases, integration strategies, and community dynamics.
MODULAR TOOLS, REUSABLE LIBRARIES, RAPID INNOVATION

LLVM

Compiler Infrastructure

llvm.org


LLVM:
THE UNIVERSAL
CODE ENGINE
C/C++ (Clang)
SYSTEMS & PERFORMANCE
RUST
SAFETY & CONCURRENCY
SWIFT
APP DEVELOPMENT
JULIA
SCIENTIFIC COMPUTING
ZIG
ROBUST & OPTIMIZED
OTHER
FRONTENDS
LLVM CORE
(IR)
LANGUAGE-AGNOSTIC
INTERMEDIATE REPRESENTATION
& OPTIMIZATION PIPELINE
x86/x64
DESKTOP & SERVER
ARM
MOBILE & EMBEDDED
WEBASSEMBLY
WEB BROWSERS
GPU (NVPTX/AMDGPU)
PARALLEL PROCESSING
RISC-V
OPEN ISA
OTHER
BACKENDS
> RUN LLVM_OVERVIEW.EXE... SYSTEM READY.

CLANG & THE C FAMILY: COMPILER, PLATFORMS, FEATURES

C FAMILY LANGUAGES

C
(Procedural Base)

C++
(Object-Oriented & Generic)

Objective-C
(Dynamic Messaging)

SUPPORTED PLATFORMS & TARGETS

DESKTOP
(x86, ARM)

MOBILE
(iOS, Android)

EMBEDDED
(RISC-V, Microcontrollers)

WEB
(WebAssembly - Wasm)

KEY CLANG FEATURES & TOOLING

FAST COMPILATION
(Parallel builds, precompiled headers)

EXPRESSIVE DIAGNOSTICS
(Clear errors, fix-its)

MODULAR ARCHITECTURE
(Library-based, extensible)

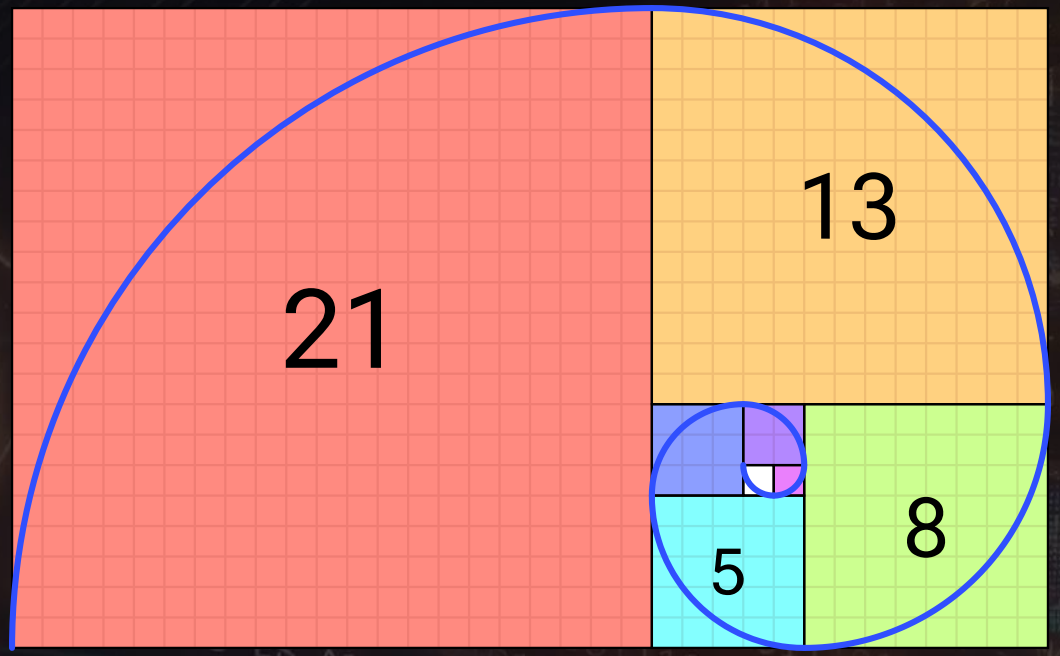
TOOLING ECOSYSTEM
(Clang-Tidy, Clang-Format, LLDB)

clang.llvm.org

Code Examples

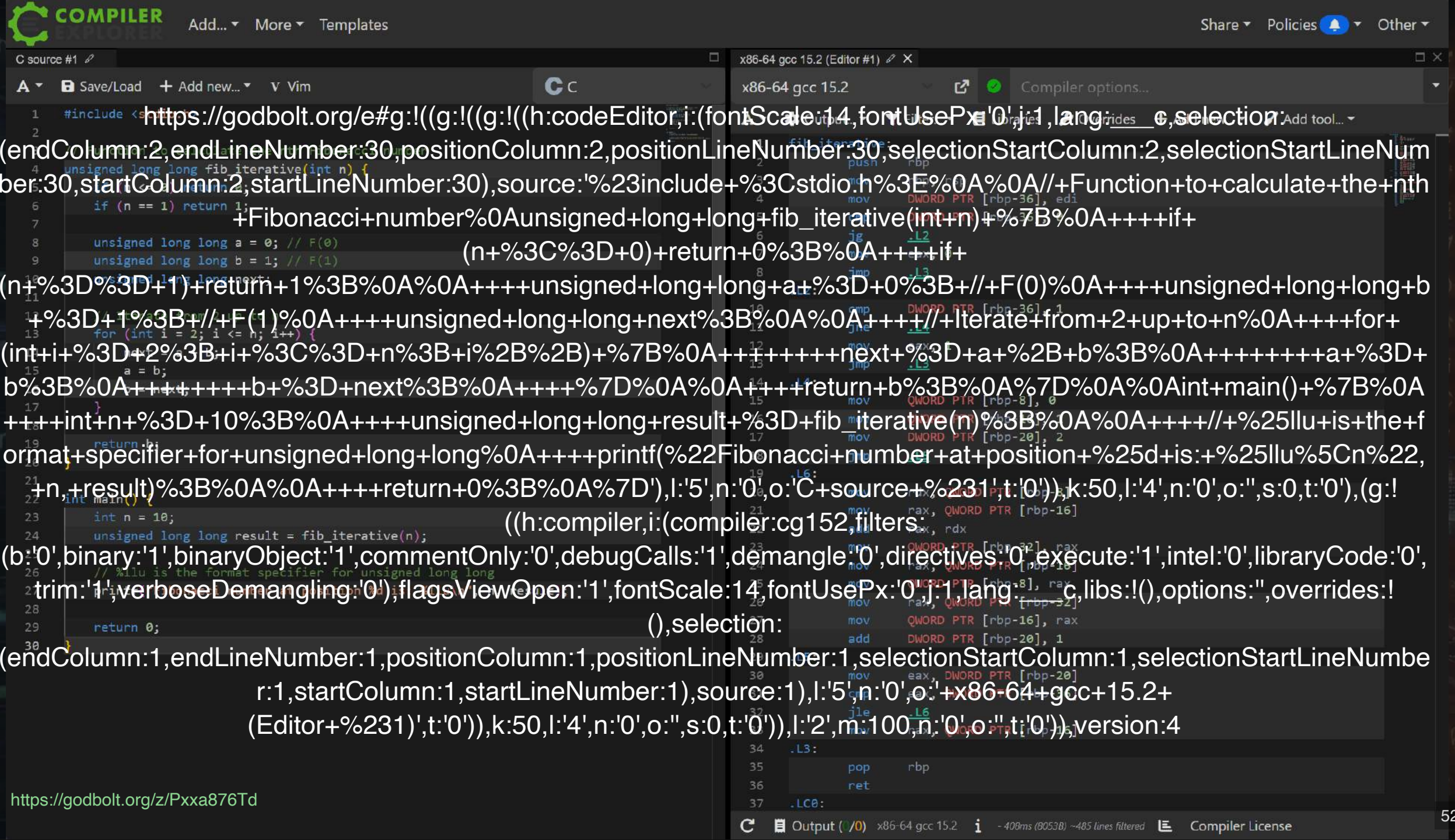
- Iterative Fibonacci sequence
- Sum of the two preceding elements
- 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, ...

$$F_0 = 0, \quad F_1 = 1,$$
$$[F_n = F_{n-1} + F_{n-2}$$
$$n > 1]$$



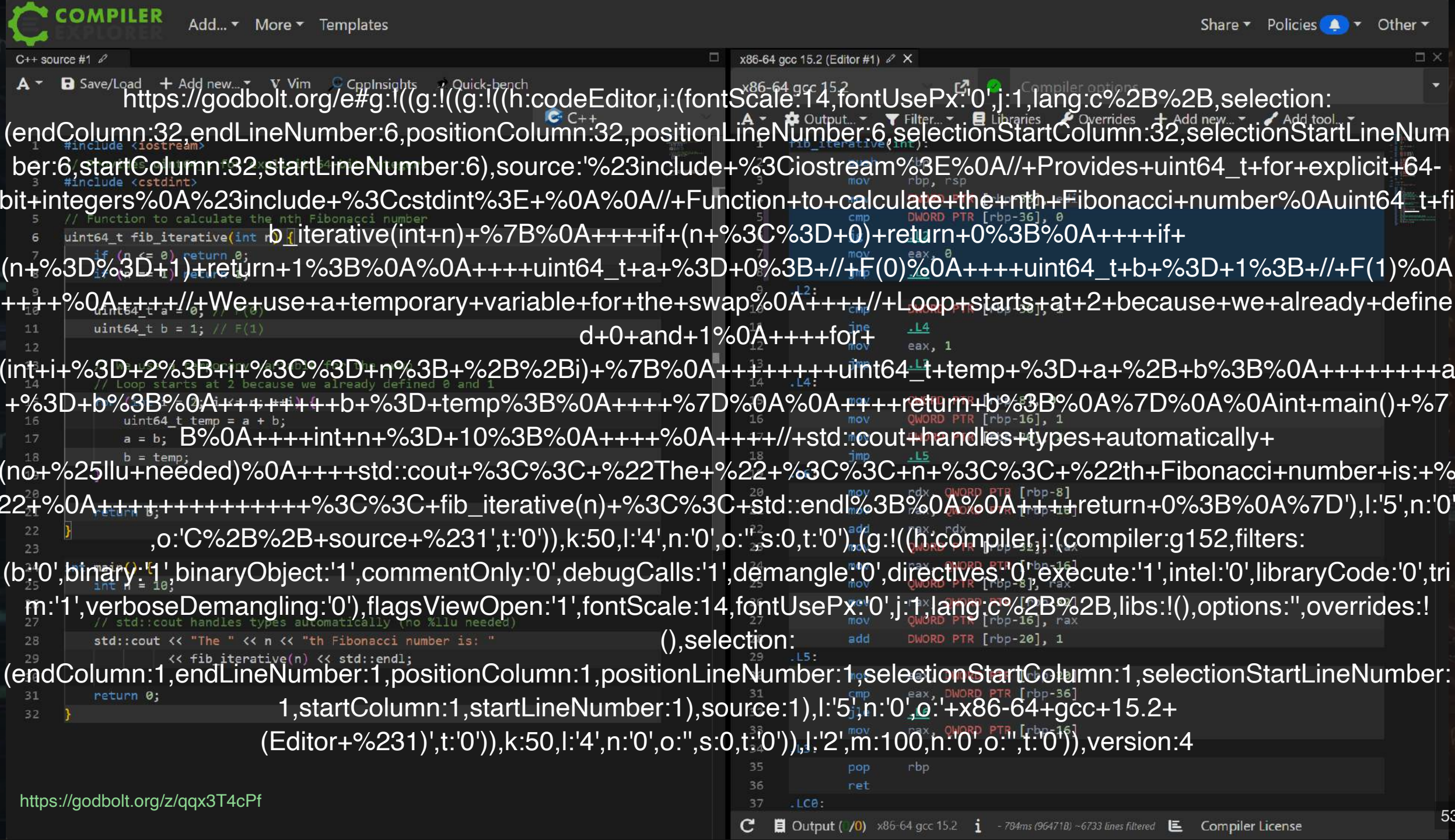
https://en.wikipedia.org/wiki/Fibonacci_sequence

Fibonacci spiral by Randaum



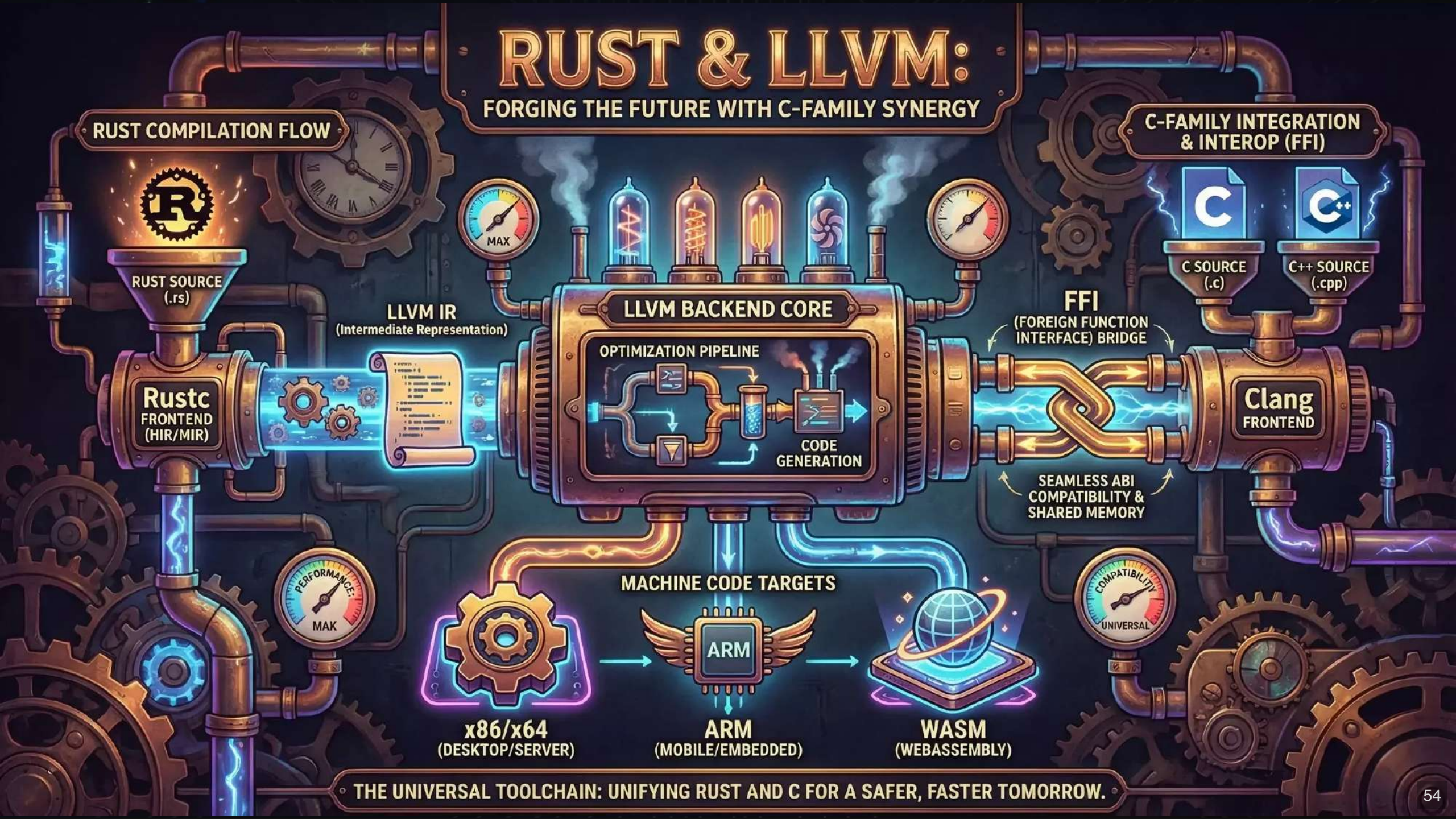
https://godbolt.org/e#g:!((g:!((g:!((h:codeEditor,i:(fontScale:14,fontUsePx:'0',j:1,lang:___c,selection:(endColumn:2,endLineNumber:30,positionColumn:2,positionLineNumber:30,selectionStartColumn:2,selectionStartLineNumber:30,startColumn:2,startLineNumber:30),source:'%23include+%3Cstdio.h%3E%0A%0A//+Function+to+calculate+the+nth+Fibonacci+number%0Aunsigned+long+long+fib_iterative(int+n)+%7B%0A++++if+(n+%3C%3D+0)+return+0%3B%0A++++if+(n+%3D%3D+1)+return+1%3B%0A%0A++++unsigned+long+long+a+%3D+0%3B+//+F(0)%0A++++unsigned+long+long+b+%3D+1%3B+//+F(1)%0A++++unsigned+long+long+next%3B%0A%0A++++//+Iterate+from+2+up+to+n%0A++++for+(int+i+%3D+2%3B+i+%3C%3D+n%3B+i%2B%2B)+%7B%0A++++++++next+%3D+a+%2B+b%3B%0A++++++++a+%3D+b%3B%0A++++++++b+%3D+next%3B%0A++++%7D%0A%0A++++return+b%3B%0A%7D%0A%0Aint+main()+%7B%0A++++int+n+%3D+10%3B%0A++++unsigned+long+long+result+%3D+fib_iterative(n)%3B%0A%0A++++//+%25llu+is+the+format+specifier+for+unsigned+long+long%0A++++printf(%22Fibonacci+number+at+position+%25d+is:+%25llu%5Cn%22,+n,+result)%3B%0A%0A++++return+0%3B%0A%7D'),l:'5',n:'0',o:'C+source+%231',t:'0')),k:50,l:'4',n:'0',o:'',s:0,t:'0'),(g:!((h:compiler,i:(compiler:cg152,filters:(b:'0',binary:'1',binaryObject:'1',commentOnly:'0',debugCalls:'1',demangle:'0',directives:'0',execute:'1',intel:'0',libraryCode:'0',trim:'1',verboseDemangling:'0'),flagsViewOpen:'1',fontScale:14,fontUsePx:'0',j:1,lang:___c,libs:!(),options:'',overrides:!(),selection:(endColumn:1,endLineNumber:1,positionColumn:1,positionLineNumber:1,selectionStartColumn:1,selectionStartLineNumber:1,startColumn:1,startLineNumber:1),source:1),l:'5',n:'0',o:'+x86-64+gcc+15.2+(Editor+%231)',t:'0')),k:50,l:'4',n:'0',o:'',s:0,t:'0')),l:'2',m:100,n:'0',o:'',t:'0')),version:4

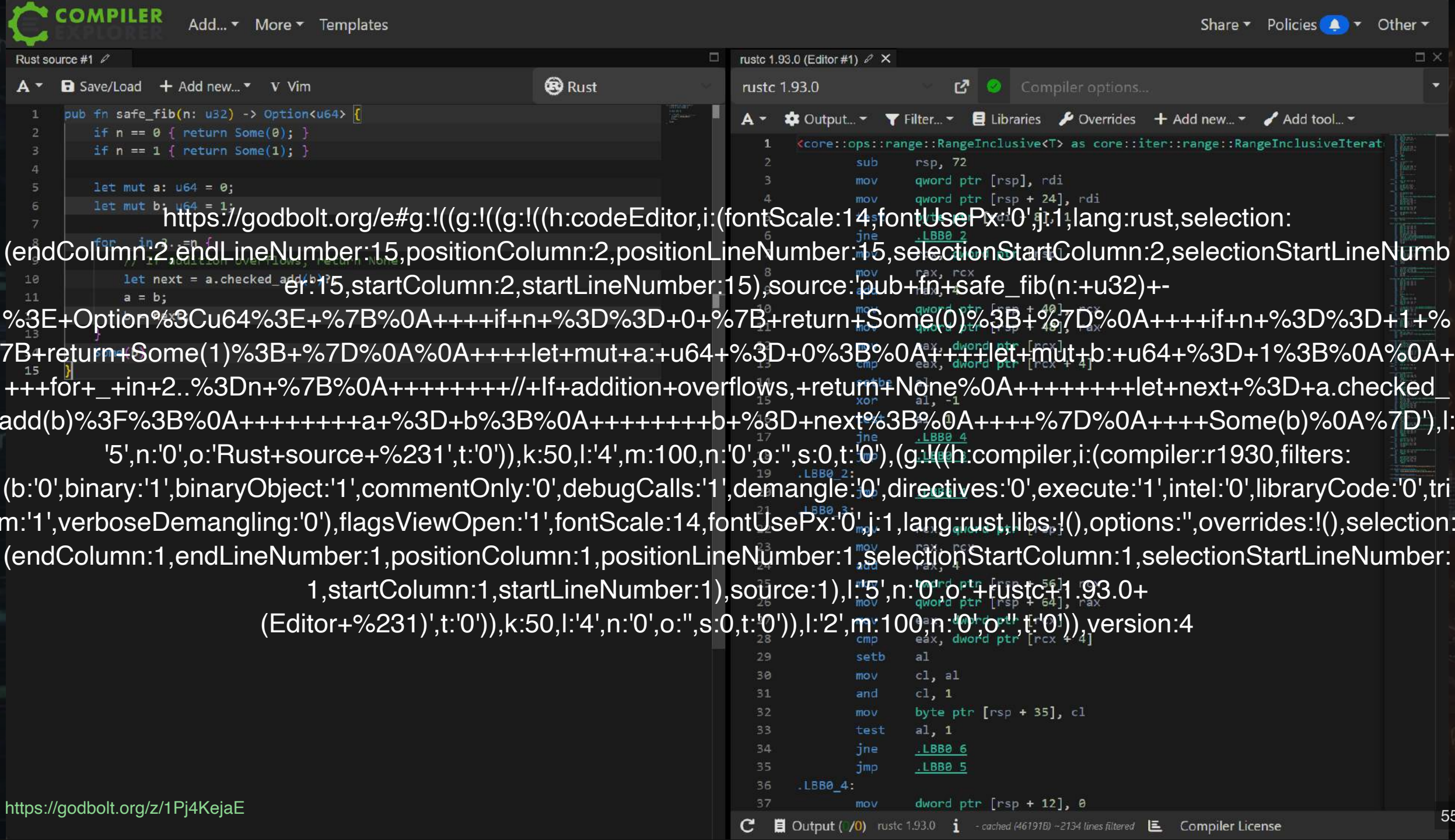
https://godbolt.org/z/Pxxa876Td



https://godbolt.org/e#g:!((g:!((g:!((h:codeEditor,i:(fontScale:14,fontUsePx:'0',j:1,lang:c%2B%2B,selection:(endColumn:32,endLineNumber:6,positionColumn:32,positionLineNumber:6,selectionStartColumn:32,selectionStartLineNumber:6,startColumn:32,startLineNumber:6),source:'%23include+%3Ciostream%3E%0A//+Provides+uint64_t+for+explicit+64-bit+integers%0A%23include+%3Ccstdint%3E+%0A%0A//+Function+to+calculate+the+nth+Fibonacci+number%0Auint64_t+fib_iterative(int+n)+%7B%0A++++if+(n+%3C%3D+0)+return+0%3B%0A++++if+(n+%3D%3D+1)+return+1%3B%0A%0A++++uint64_t+a+%3D+0%3B+//+F(0)%0A++++uint64_t+b+%3D+1%3B+//+F(1)%0A++++%0A++++//+We+use+a+temporary+variable+for+the+swap%0A++++//+Loop+starts+at+2+because+we+already+defined+0+and+1%0A++++for+(int+i+%3D+2%3B+i+%3C%3D+n%3B+%2B%2Bi)+%7B%0A++++++++uint64_t+temp+%3D+a+%2B+b%3B%0A++++++++a+%3D+b%3B%0A++++++++b+%3D+temp%3B%0A++++%7D%0A%0A++++return+b%3B%0A%7D%0A%0Aint+main()+%7B%0A++++int+n+%3D+10%3B%0A++++%0A++++//+std::cout+handles+types+automatically+(no+%25llu+needed)%0A++++std::cout+%3C%3C+%22The+%22+%3C%3C+n+%3C%3C+%22th+Fibonacci+number+is:+%22+%0A++++++++++++%3C%3C+fib_iterative(n)+%3C%3C+std::endl%3B%0A%0A++++return+0%3B%0A%7D'),l:'5',n:'0',o:'C%2B%2B+source+%231',t:'0')),k:50,l:'4',n:'0',o:'',s:0,t:'0'),(g:!((h:compiler,i:(compiler:g152,filters:(b:'0',binary:'1',binaryObject:'1',commentOnly:'0',debugCalls:'1',demangle:'0',directives:'0',execute:'1',intel:'0',libraryCode:'0',trim:'1',verboseDemangling:'0'),flagsViewOpen:'1',fontScale:14,fontUsePx:'0',j:1,lang:c%2B%2B,libs:!(),options:'',overrides:!(),selection:(endColumn:1,endLineNumber:1,positionColumn:1,positionLineNumber:1,selectionStartColumn:1,selectionStartLineNumber:1,startColumn:1,startLineNumber:1),source:1),l:'5',n:'0',o:'+x86-64+gcc+15.2+(Editor+%231)',t:'0')),k:50,l:'4',n:'0',o:'',s:0,t:'0')),l:'2',m:100,n:'0',o:'',t:'0')),version:4

https://godbolt.org/z/qqx3T4cPf


RUST & LLVM:
FORGING THE FUTURE WITH C-FAMILY SYNERGY
RUST COMPILATION FLOW
RUST SOURCE (.rs)
LLVM IR (Intermediate Representation)
Rustc FRONTEND (HIR/MIR)
LLVM BACKEND CORE
OPTIMIZATION PIPELINE
CODE GENERATION
MAX
C-FAMILY INTEGRATION & INTEROP (FFI)
C SOURCE (.c)
C++ SOURCE (.cpp)
FFI (FOREIGN FUNCTION INTERFACE) BRIDGE
Clang FRONTEND
SEAMLESS ABI COMPATIBILITY & SHARED MEMORY
PERFORMANCE
MAK
COMPATIBILITY
UNIVERSAL
MACHINE CODE TARGETS
ARM
x86/x64 (DESKTOP/SERVER)
ARM (MOBILE/EMBEDDED)
WASM (WEBASSEMBLY)
THE UNIVERSAL TOOLCHAIN: UNIFYING RUST AND C FOR A SAFER, FASTER TOMORROW.



https://godbolt.org/e#g:!((g:!((g:!((h:codeEditor,i:(fontScale:14,fontUsePx:'0',j:1,lang:rust,selection:(endColumn:2,endLineNumber:15,positionColumn:2,positionLineNumber:15,selectionStartColumn:2,selectionStartLineNumber:15,startColumn:2,startLineNumber:15),source:'pub+fn+safe_fib(n:+u32)+-%3E+Option%3Cu64%3E+%7B%0A++++if+n+%3D%3D+0+%7B+return+Some(0)%3B+%7D%0A++++if+n+%3D%3D+1+%7B+return+Some(1)%3B+%7D%0A%0A++++let+mut+a:+u64+%3D+0%3B%0A++++let+mut+b:+u64+%3D+1%3B%0A%0A++++for+_+in+2..%3Dn+%7B%0A++++++++//+If+addition+overflows,+return+None%0A++++++++let+next+%3D+a.checked_add(b)%3F%3B%0A++++++++a+%3D+b%3B%0A++++++++b+%3D+next%3B%0A++++%7D%0A++++Some(b)%0A%7D'),l:'5',n:'0',o:'Rust+source+%231',t:'0')),k:50,l:'4',m:100,n:'0',o:'',s:0,t:'0'),(g:!((h:compiler,i:(compiler:r1930,filters:(b:'0',binary:'1',binaryObject:'1',commentOnly:'0',debugCalls:'1',demangle:'0',directives:'0',execute:'1',intel:'0',libraryCode:'0',trim:'1',verboseDemangling:'0'),flagsViewOpen:'1',fontScale:14,fontUsePx:'0',j:1,lang:rust,libs:!(),options:'',overrides:!(),selection:(endColumn:1,endLineNumber:1,positionColumn:1,positionLineNumber:1,selectionStartColumn:1,selectionStartLineNumber:1,startColumn:1,startLineNumber:1),source:1),l:'5',n:'0',o:'+rustc+1.93.0+(Editor+%231)',t:'0')),k:50,l:'4',n:'0',o:'',s:0,t:'0')),l:'2',m:100,n:'0',o:'',t:'0')),version:4

https://godbolt.org/z/1Pj4KejaE

KOTLIN: THE INDUSTRIAL-GRADE LANGUAGE

CONCISENESS & READABILITY
(Less Boilerplate, More Power)

OLD JAVA CODE

Streamlined syntax. Reduces verbosity.

KOTLIN CORE ENGINE

MULTIPLATFORM
(Code Sharing Across Platforms)

Share logic between Mobile, Web, & Native.

INTEROPERABILITY
(Seamless Java Integration)

Works with existing Java libraries & frameworks.

NULL SAFETY
(Preventing the Billion-Dollar Mistake)

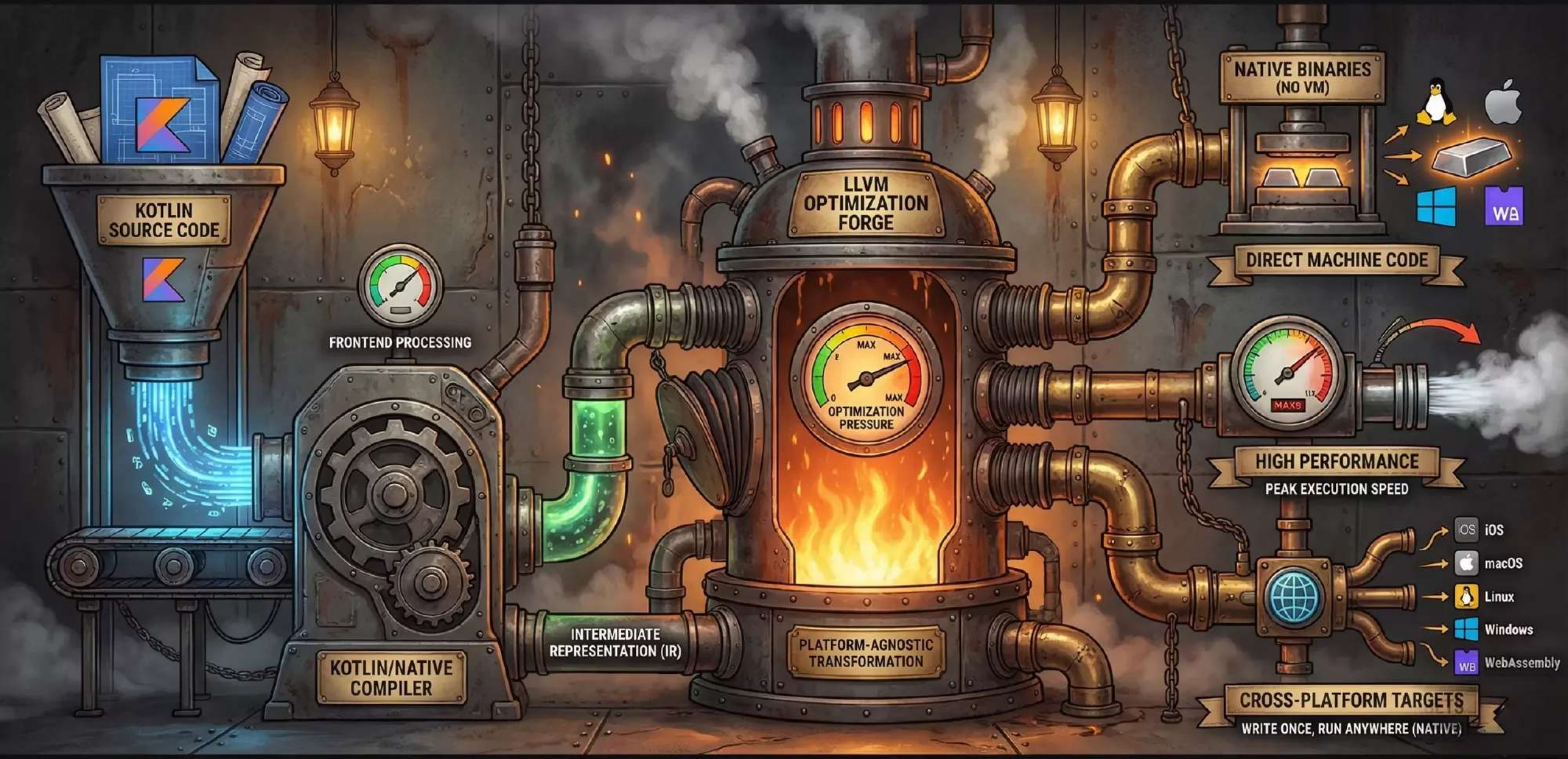
Eliminates NullPointerExceptions at compile time.

COROUTINES
(Asynchronous Programming)

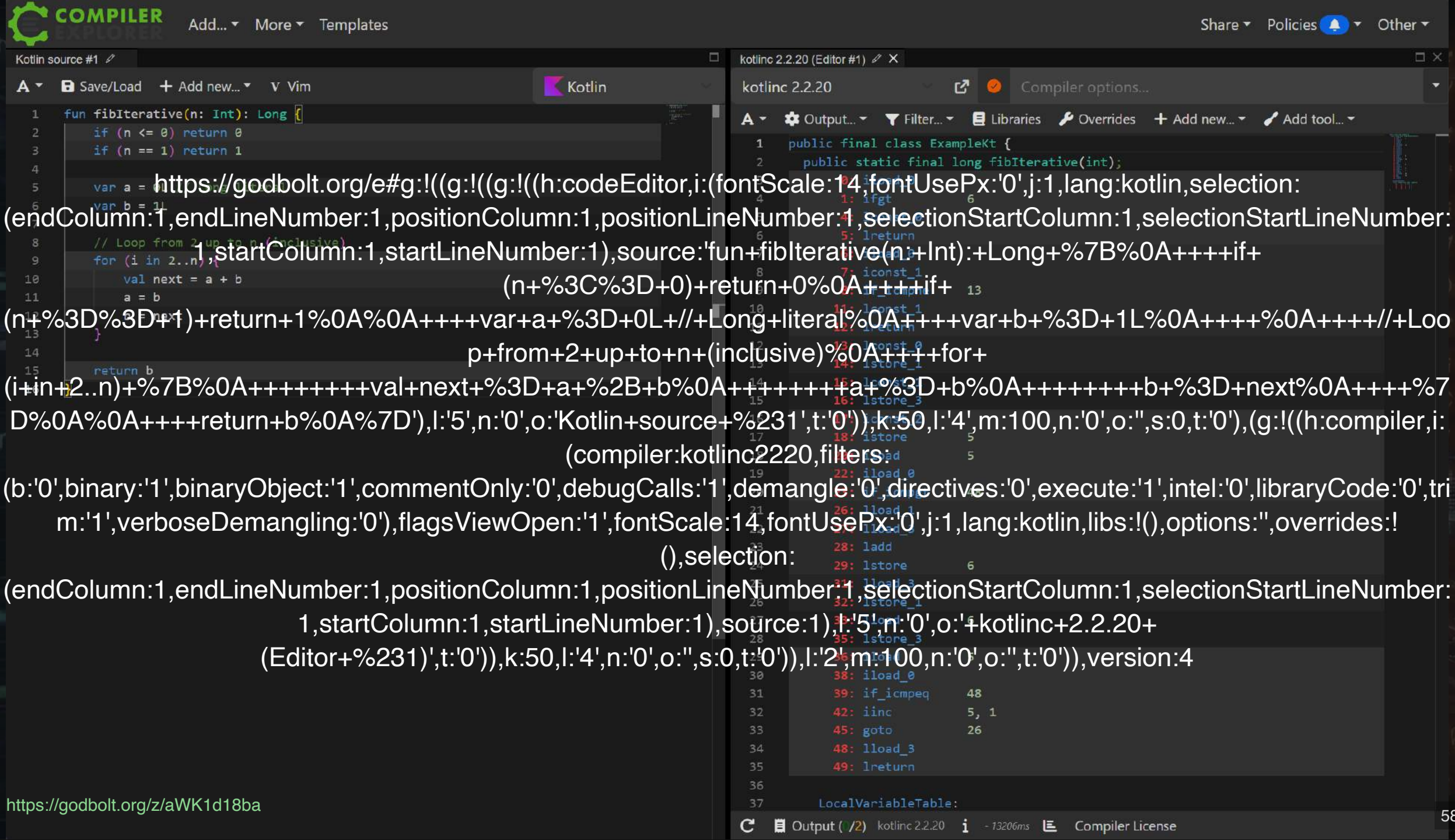
Simplify concurrent code. Non-blocking operations.

MODERN. PRAGMATIC. SAFE. THE ENGINE OF FUTURE DEVELOPMENT.

KOTLIN & LLVM: FORGING NATIVE POWER



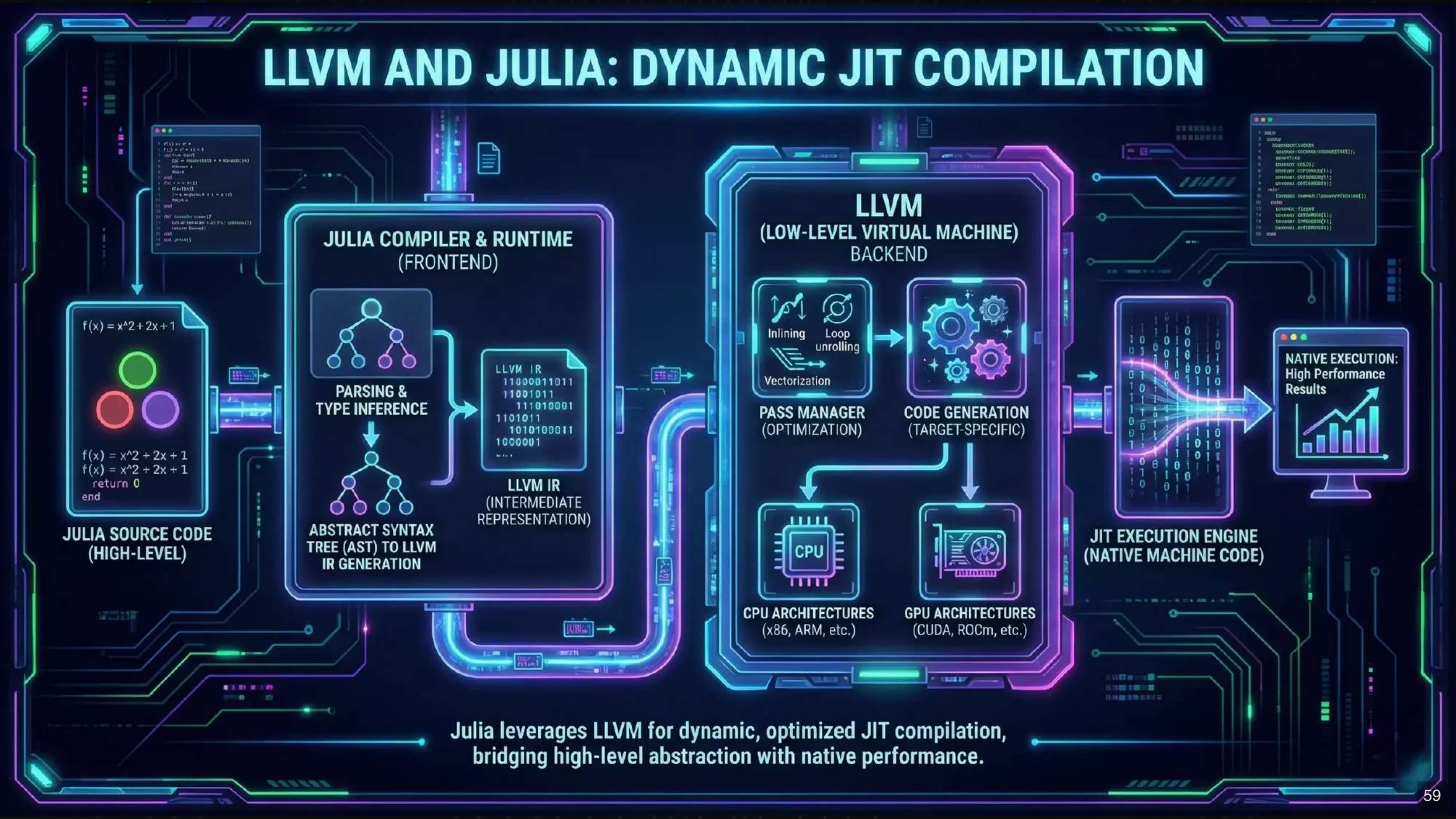
Kotlin/Native leverages the LLVM toolchain to compile directly to standalone native binaries, unlocking high performance and broad platform reach without a virtual machine



https://godbolt.org/e#g:!((g:!((g:!((h:codeEditor,i:(fontScale:14,fontUsePx:'0',j:1,lang:kotlin,selection:(endColumn:1,endLineNumber:1,positionColumn:1,positionLineNumber:1,selectionStartColumn:1,selectionStartLineNumber:1,startColumn:1,startLineNumber:1),source:'fun+fibIterative(n:+Int):+Long+%7B%0A++++if+(n+%3C%3D+0)+return+0%0A++++if+(n+%3D%3D+1)+return+1%0A%0A++++var+a+%3D+0L+//+Long+literal%0A++++var+b+%3D+1L%0A++++%0A++++//+Loop+from+2+up+to+n+(inclusive)%0A++++for+(i+in+2..n)+%7B%0A++++++++val+next+%3D+a+%2B+b%0A++++++++a+%3D+b%0A++++++++b+%3D+next%0A++++%7D%0A%0A++++return+b%0A%7D'),l:'5',n:'0',o:'Kotlin+source+%231',t:'0')),k:50,l:'4',m:100,n:'0',o:'',s:0,t:'0'),(g:!((h:compiler,i:(compiler:kotlinc2220,filters:(b:'0',binary:'1',binaryObject:'1',commentOnly:'0',debugCalls:'1',demangle:'0',directives:'0',execute:'1',intel:'0',libraryCode:'0',trim:'1',verboseDemangling:'0'),flagsViewOpen:'1',fontScale:14,fontUsePx:'0',j:1,lang:kotlin,libs:!(),options:'',overrides:!(),selection:(endColumn:1,endLineNumber:1,positionColumn:1,positionLineNumber:1,selectionStartColumn:1,selectionStartLineNumber:1,startColumn:1,startLineNumber:1),source:1),l:'5',n:'0',o:'+kotlinc+2.2.20+(Editor+%231)',t:'0')),k:50,l:'4',n:'0',o:'',s:0,t:'0')),l:'2',m:100,n:'0',o:'',t:'0')),version:4

https://godbolt.org/z/aWK1d18ba

LLVM AND JULIA: DYNAMIC JIT COMPILATION



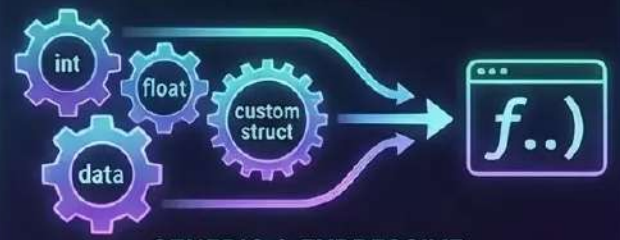
Julia leverages LLVM for dynamic, optimized JIT compilation,
bridging high-level abstraction with native performance.

JULIA: HIGH-PERFORMANCE DYNAMIC PROGRAMMING

JUST-IN-TIME COMPILATION (LLVM)

NEAR-NATIVE SPEED.
COMPILS TO MACHINE CODE.

MULTIPLE DISPATCH



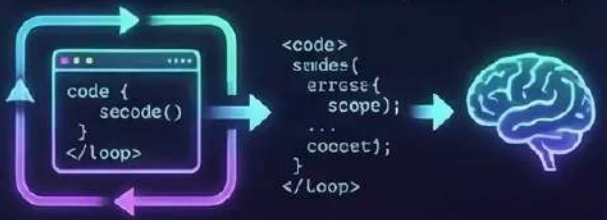
GENERIC & EXPRESSIVE.
EFFICIENT FUNCTION SELECTION.

DYNAMIC TYPING

NUMBER STRING ARRAY

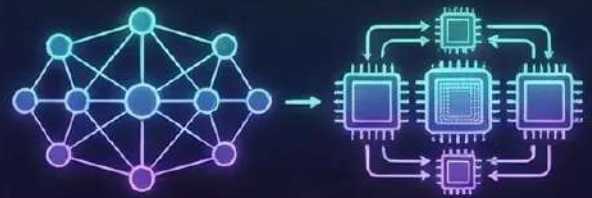
FLEXIBLE & INTERACTIVE.
RAPID PROTOTYPING.

METAPROGRAMMING (MACROS)



CODE THAT WRITES CODE.
POWERFUL ABSTRACTIONS.

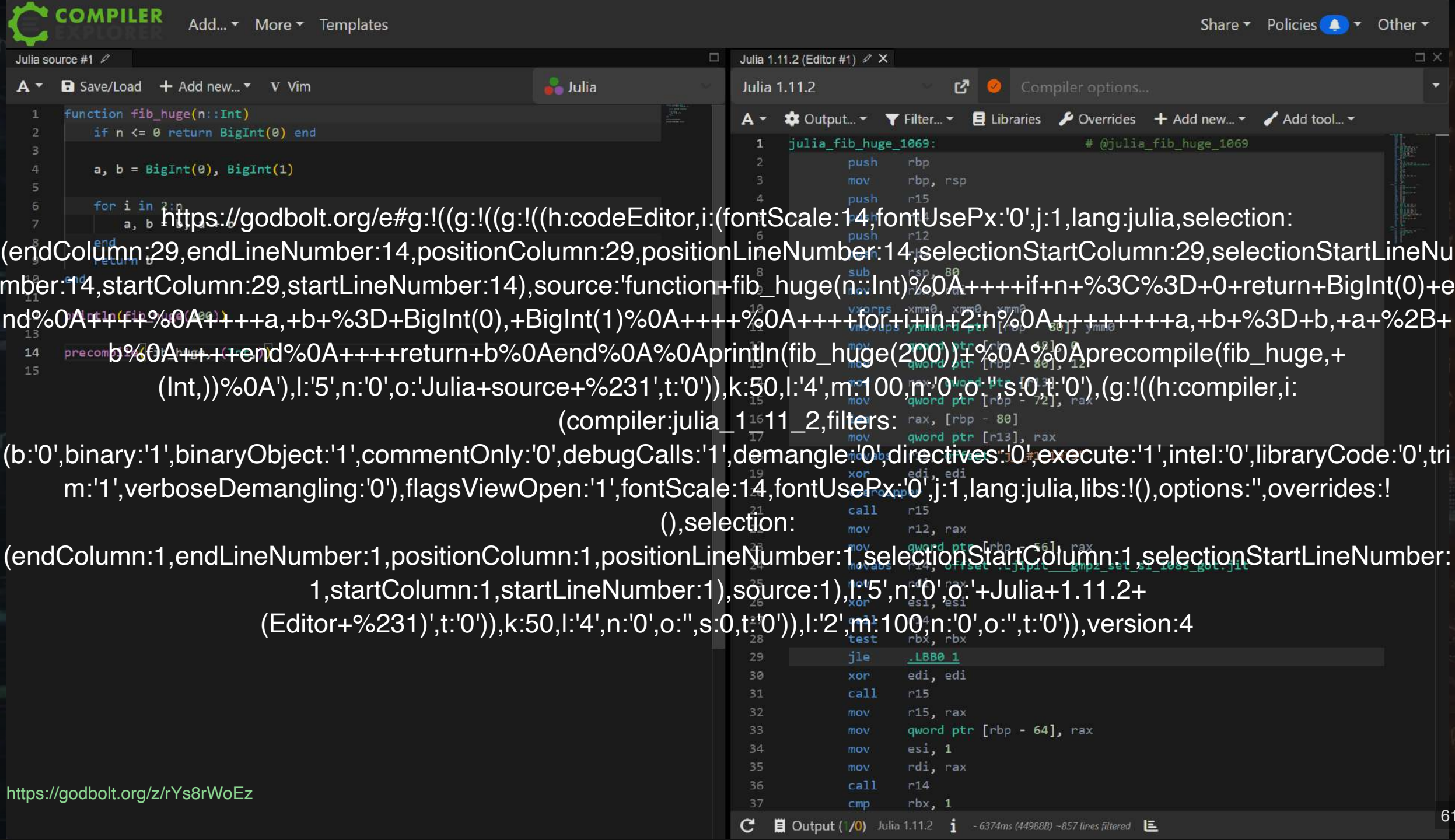
PARALLEL & DISTRIBUTED COMPUTING



SCALABLE PERFORMANCE.
EASY CONCURRENCY.

BUILT-FOR-MATH & DATA SCIENCE

EFFICIENT NUMERICAL LIBRARIES.
FIRST-CLASS ARRAY SUPPORT.



https://godbolt.org/e#g:!((g:!((g:!((h:codeEditor,i:(fontScale:14,fontUsePx:'0',j:1,lang:julia,selection:(endColumn:29,endLineNumber:14,positionColumn:29,positionLineNumber:14,selectionStartColumn:29,selectionStartLineNumber:14,startColumn:29,startLineNumber:14),source:'function+fib_huge(n::Int)%0A++++if+n+%3C%3D+0+return+BigInt(0)+end%0A++++%0A++++a,+b+%3D+BigInt(0),+BigInt(1)%0A++++%0A++++for+i+in+2:n%0A++++++++a,+b+%3D+b,+a+%2B+b%0A++++end%0A++++return+b%0Aend%0A%0Aprintln(fib_huge(200))+%0A%0Aprecompile(fib_huge,+(Int,))%0A'),l:'5',n:'0',o:'Julia+source+%231',t:'0')),k:50,l:'4',m:100,n:'0',o:'',s:0,t:'0'),(g:!((h:compiler,i:(compiler:julia_1_11_2,filters:(b:'0',binary:'1',binaryObject:'1',commentOnly:'0',debugCalls:'1',demangle:'0',directives:'0',execute:'1',intel:'0',libraryCode:'0',trim:'1',verboseDemangling:'0'),flagsViewOpen:'1',fontScale:14,fontUsePx:'0',j:1,lang:julia,libs:!(),options:'',overrides:!(),selection:(endColumn:1,endLineNumber:1,positionColumn:1,positionLineNumber:1,selectionStartColumn:1,selectionStartLineNumber:1,startColumn:1,startLineNumber:1),source:1),l:'5',n:'0',o:'+Julia+1.11.2+(Editor+%231)',t:'0')),k:50,l:'4',n:'0',o:'',s:0,t:'0')),l:'2',m:100,n:'0',o:'',t:'0')),version:4

https://godbolt.org/z/rYs8rWoEz

LLVM AND SWIFT: EXTENDING & ENABLING

SWIFT SOURCE
(FRONTEND)

```
func greet(name: String) {
  print("Hello, \(name)!")
}
```

SIL GENERATION & HIGH-LEVEL OPTIMIZATION

SWIFT INTERMEDIATE LANGUAGE (SIL)

LLVM CORE
(MIDDLE-END)

LLVM IR GENERATION

LLVM OPTIMIZER (PASS MANAGER)

GENERAL-PURPOSE OPTIMIZATIONS (INLINING, VECTORIZATION, ETC.)

LLVM BACKEND
(TARGET GENERATION - APP STORE)

iOS (ARM64) · macOS (Apple Silicon) · watchOS · tvOS · App Store

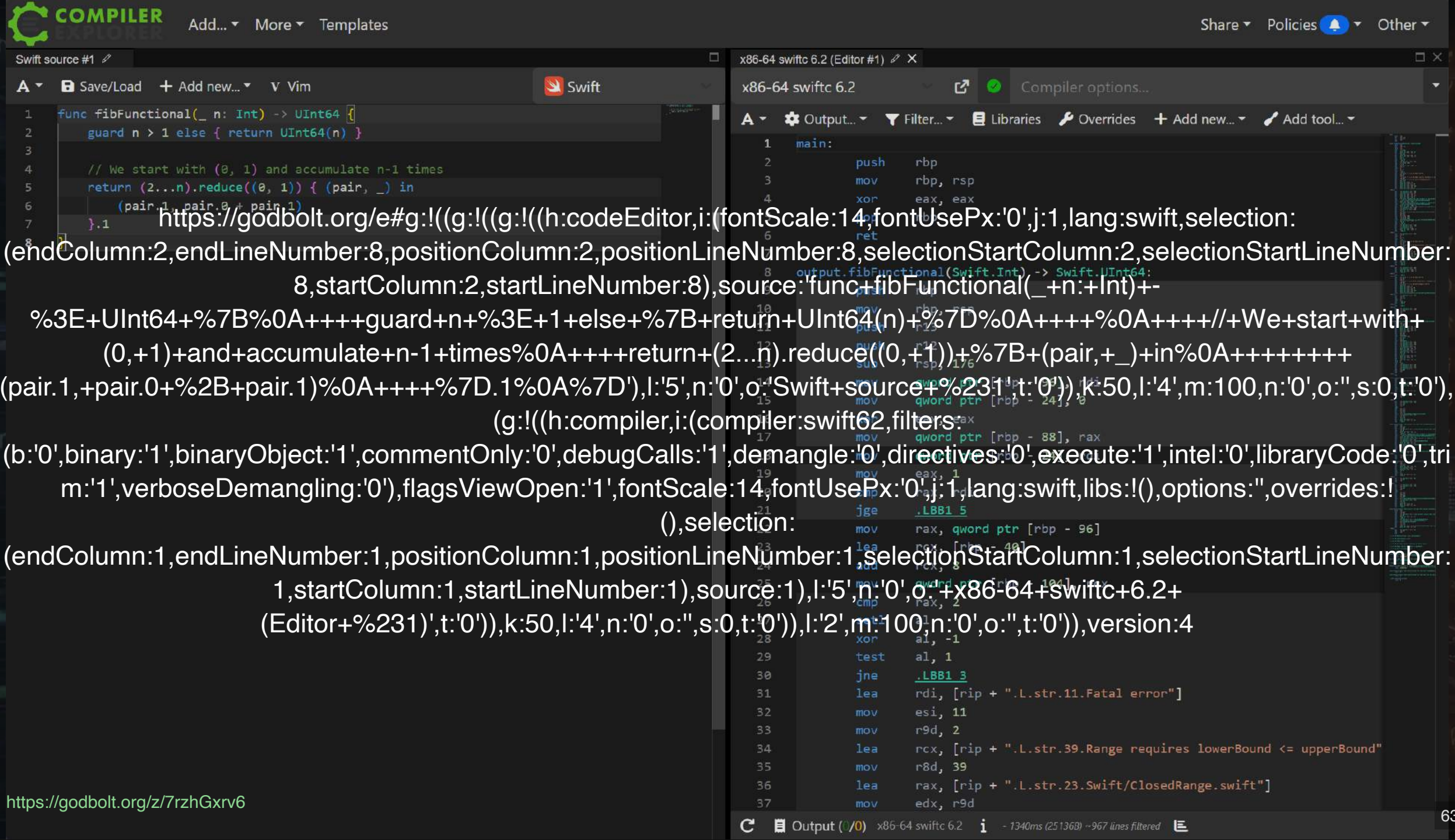
LLVM BACKEND
(TARGET GENERATION - CROSS-PLATFORM)

LINUX (x86-64, ARM64) · WINDOWS (x86-64, ARM64) · ANDROID (ARM64)

LLVM BACKEND
(TARGET GENERATION - WEB)

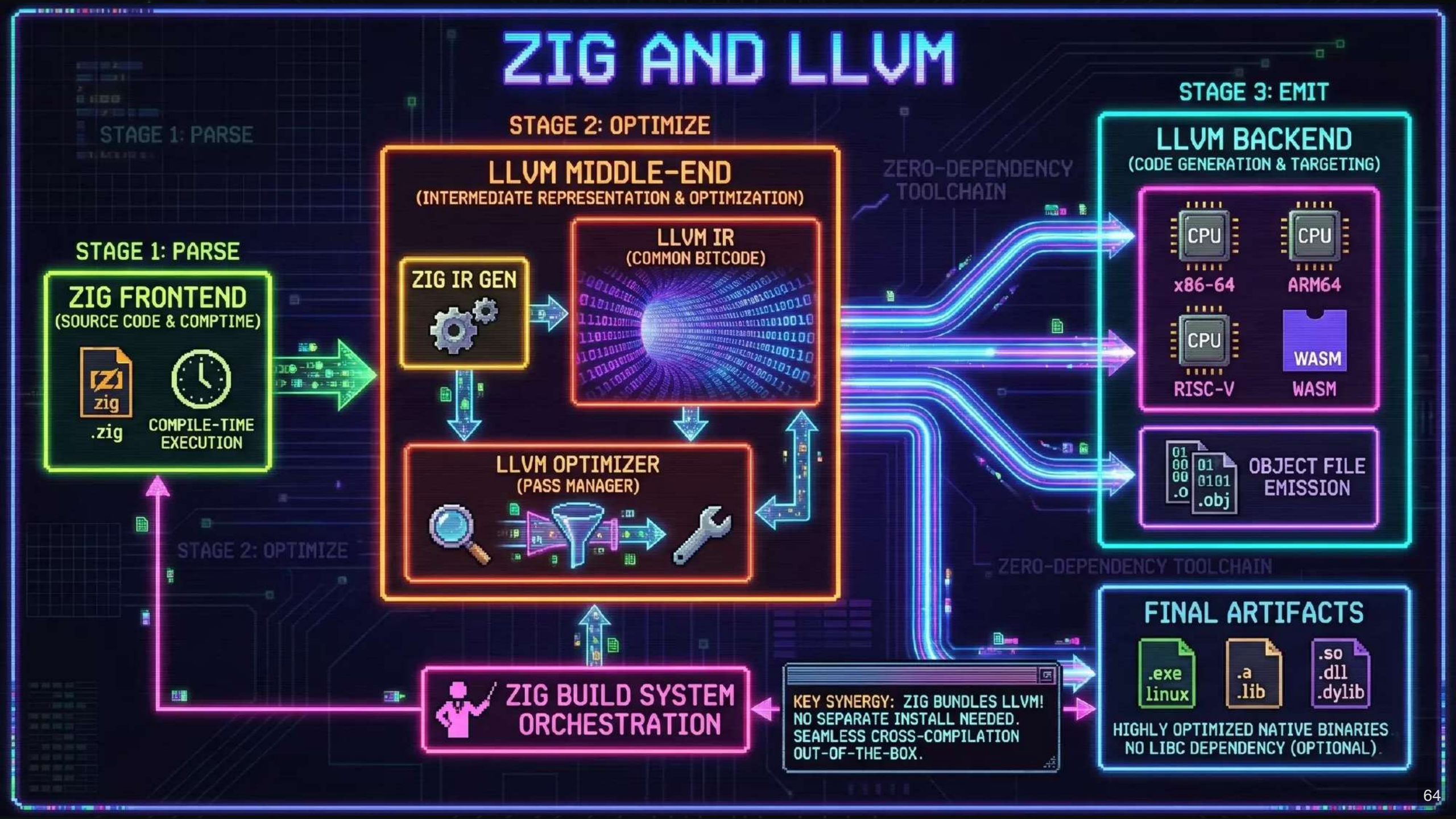
WebAssembly (WASM)

KEY ENABLER: LLVM PROVIDES CROSS-PLATFORM REACH & ROBUST OPTIMIZATION FOR SWIFT.



https://godbolt.org/e#g:!((g:!((g:!((h:codeEditor,i:(fontScale:14,fontUsePx:'0',j:1,lang:swift,selection:(endColumn:2,endLineNumber:8,positionColumn:2,positionLineNumber:8,selectionStartColumn:2,selectionStartLineNumber:8,startColumn:2,startLineNumber:8),source:'func+fibFunctional(_+n:+Int)+-%3E+UInt64+%7B%0A++++guard+n+%3E+1+else+%7B+return+UInt64(n)+%7D%0A++++%0A++++//+We+start+with+(0,+1)+and+accumulate+n-1+times%0A++++return+(2...n).reduce((0,+1))+%7B+(pair,+_)+in%0A++++++++(pair.1,+pair.0+%2B+pair.1)%0A++++%7D.1%0A%7D'),l:'5',n:'0',o:'Swift+source+%231',t:'0')),k:50,l:'4',m:100,n:'0',o:'',s:0,t:'0'),(g:!((h:compiler,i:(compiler:swift62,filters:(b:'0',binary:'1',binaryObject:'1',commentOnly:'0',debugCalls:'1',demangle:'0',directives:'0',execute:'1',intel:'0',libraryCode:'0',trim:'1',verboseDemangling:'0'),flagsViewOpen:'1',fontScale:14,fontUsePx:'0',j:1,lang:swift,libs:!(),options:'',overrides:!(),selection:(endColumn:1,endLineNumber:1,positionColumn:1,positionLineNumber:1,selectionStartColumn:1,selectionStartLineNumber:1,startColumn:1,startLineNumber:1),source:1),l:'5',n:'0',o:'+x86-64+swiftc+6.2+(Editor+%231)',t:'0')),k:50,l:'4',n:'0',o:'',s:0,t:'0')),l:'2',m:100,n:'0',o:'',t:'0')),version:4

https://godbolt.org/z/7rzhGxrv6


ZIG AND LLVM
STAGE 1: PARSE
STAGE 2: OPTIMIZE
STAGE 3: EMIT
LLVM MIDDLE-END
(INTERMEDIATE REPRESENTATION & OPTIMIZATION)
LLVM IR
(COMMON BITCODE)
ZIG IR GEN
LLVM OPTIMIZER
(PASS MANAGER)
ZIG FRONTEND
(SOURCE CODE & COMPTIME)
.zig
COMPILE-TIME EXECUTION
LLVM BACKEND
(CODE GENERATION & TARGETING)
CPU
x86-64
CPU
ARM64
CPU
RISC-V
WASM
WASM
OBJECT FILE EMISSION
.o
.obj
ZERO-DEPENDENCY TOOLCHAIN
ZERO-DEPENDENCY TOOLCHAIN
FINAL ARTIFACTS
.exe
linux
.a
.lib
.so
.dll
.dylib
HIGHLY OPTIMIZED NATIVE BINARIES
NO LIBC DEPENDENCY (OPTIONAL).
ZIG BUILD SYSTEM ORCHESTRATION
KEY SYNERGY: ZIG BUNDLES LLVM!
NO SEPARATE INSTALL NEEDED.
SEAMLESS CROSS-COMPILATION
OUT-OF-THE-BOX.

ZIG: SIMPLICITY, PERFORMANCE, AND SAFETY

SIMPLICITY

NO HIDDEN CONTROL FLOW

MANUAL MEMORY MANAGEMENT (allocators)

NO PREPROCESSOR MAGIC

Clear, explicit code. Control is yours.

LLVM BACKEND (CROSS-PLATFORM POWERHOUSE)

PERFORMANCE

C-COMPATIBLE & FASTER

DIRECT LLVM IR GENERATION

COMPTIME: CODE AT COMPILE-TIME

Leverages LLVM for optimized, native binaries.

SAFETY

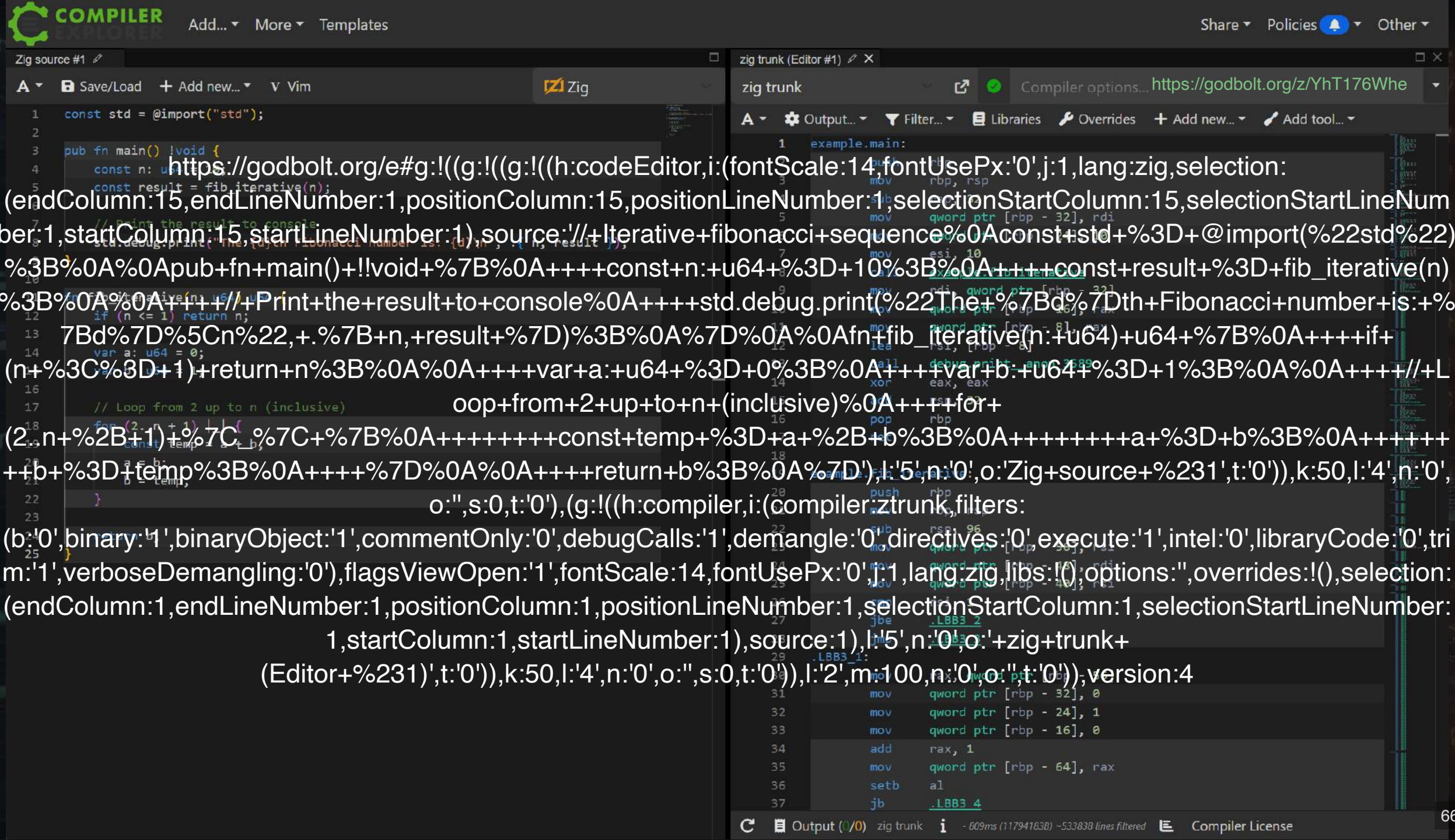
OPTIONAL TYPES (?T)

CHECKED ARITHMETIC

COMPILE-TIME CHECKS

Catch errors early. Robust and reliable.

> ZIG BUILD RUN --RELEASE-FAST.EXE // READY.

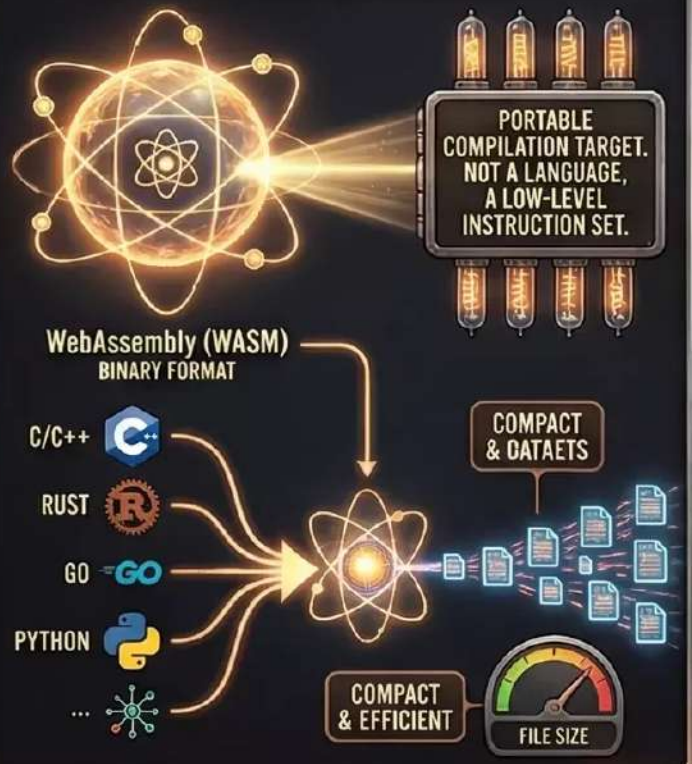


https://godbolt.org/e#g:!((g:!((g:!((h:codeEditor,i:(fontScale:14,fontUsePx:'0',j:1,lang:zig,selection:(endColumn:15,endLineNumber:1,positionColumn:15,positionLineNumber:1,selectionStartColumn:15,selectionStartLineNumber:1,startColumn:15,startLineNumber:1),source:'//+Iterative+fibonacci+sequence%0Aconst+std+%3D+@import(%22std%22)%3B%0A%0Apub+fn+main()+!!void+%7B%0A++++const+n:+u64+%3D+10%3B%0A++++const+result+%3D+fib_iterative(n)%3B%0A%0A++++//+Print+the+result+to+console%0A++++std.debug.print(%22The+%7Bd%7Dth+Fibonacci+number+is:+%7Bd%7D%5Cn%22,+.%7B+n,+result+%7D)%3B%0A%7D%0A%0Afn+fib_iterative(n:+u64)+u64+%7B%0A++++if+(n+%3C%3D+1)+return+n%3B%0A%0A++++var+a:+u64+%3D+0%3B%0A++++var+b:+u64+%3D+1%3B%0A%0A++++//+Loop+from+2+up+to+n+(inclusive)%0A++++for+(2..n+%2B+1)+%7C_%7C+%7B%0A++++++++const+temp+%3D+a+%2B+b%3B%0A++++++++a+%3D+b%3B%0A++++++++b+%3D+temp%3B%0A++++%7D%0A%0A++++return+b%3B%0A%7D'),l:'5',n:'0',o:'Zig+source+%231',t:'0')),k:50,l:'4',n:'0',o:'',s:0,t:'0'),(g:!((h:compiler,i:(compiler:ztrunk,filters:(b:'0',binary:'1',binaryObject:'1',commentOnly:'0',debugCalls:'1',demangle:'0',directives:'0',execute:'1',intel:'0',libraryCode:'0',trim:'1',verboseDemangling:'0'),flagsViewOpen:'1',fontScale:14,fontUsePx:'0',j:1,lang:zig,libs:!(),options:'',overrides:!(),selection:(endColumn:1,endLineNumber:1,positionColumn:1,positionLineNumber:1,selectionStartColumn:1,selectionStartLineNumber:1,startColumn:1,startLineNumber:1),source:1),l:'5',n:'0',o:'+zig+trunk+(Editor+%231)',t:'0')),k:50,l:'4',n:'0',o:'',s:0,t:'0')),l:'2',m:100,n:'0',o:'',t:'0')),version:4


WebAssembly
Powered by
LLVM
C/C++
(Clang)
RUST
GO
PYTHON
(via Pyodide)
SWIFT
KOTLIN
SOURCE CODE
INPUT
THE LLVM TOOLCHAIN CORE
LLVM IR
(Intermediate
Representation)
OPTIMIZATION
PIPELINE
TARGET INDEPENDENT
CODE GEN
CROSS-PLATFORM
COMPILATION
UNIVERSAL COMPILER INFRASTRUCTURE:
Transforming diverse languages
into optimized, portable code.
THE WEBASSEMBLY
UNIVERSE
GAMING & GRAPHICS
(WebGL/WebGPU)
HIGH-PERFORMANCE
APPS
AUDIO & VIDEO
PROCESSING
MACHINE LEARNING
INFERENCE
WEB BROWSER EXPANSION
(Client-Side)
Near-native speed
in the browser,
breaking JS limits!
SERVERLESS
FUNCTIONS
MICROSERVICES
EDGE
COMPUTING
CLOUD-NATIVE
MODULES (WASI)
SERVER & CLOUD
(Server-Side)
Secure, portable, and efficient
runtime outside the browser.
IoT &
BLOCKCHAIN
SMART CONTRACTS
PLUGIN
SYSTEMS
DESKTOP
APPLICATIONS
BEYOND THE WEB
(Universal Runtime)
LLVM empowers WebAssembly
to become the universal binary
format for all platforms, driving
the future of computing

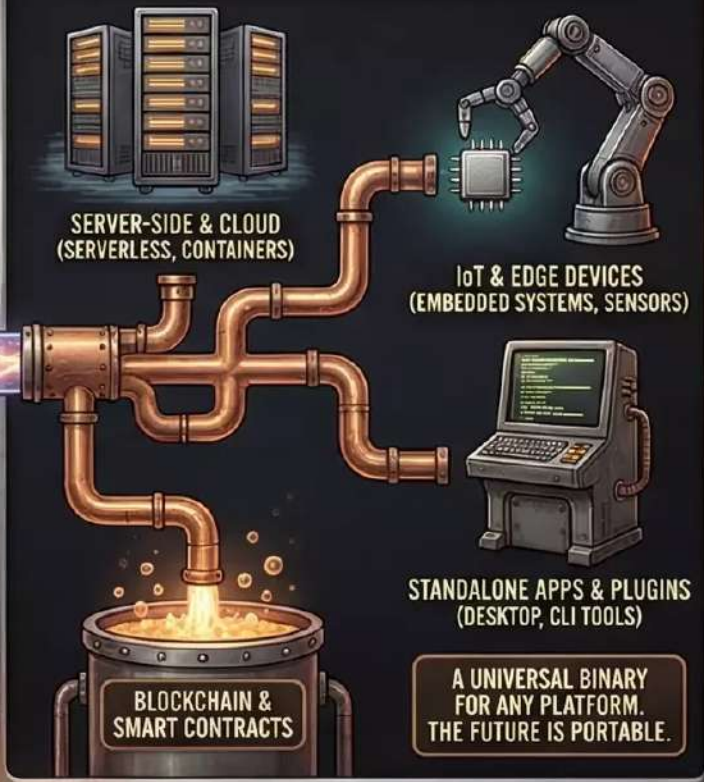
WebAssembly: More than JavaScript

1. THE ATOMIC CORE: WHAT IS WASM?

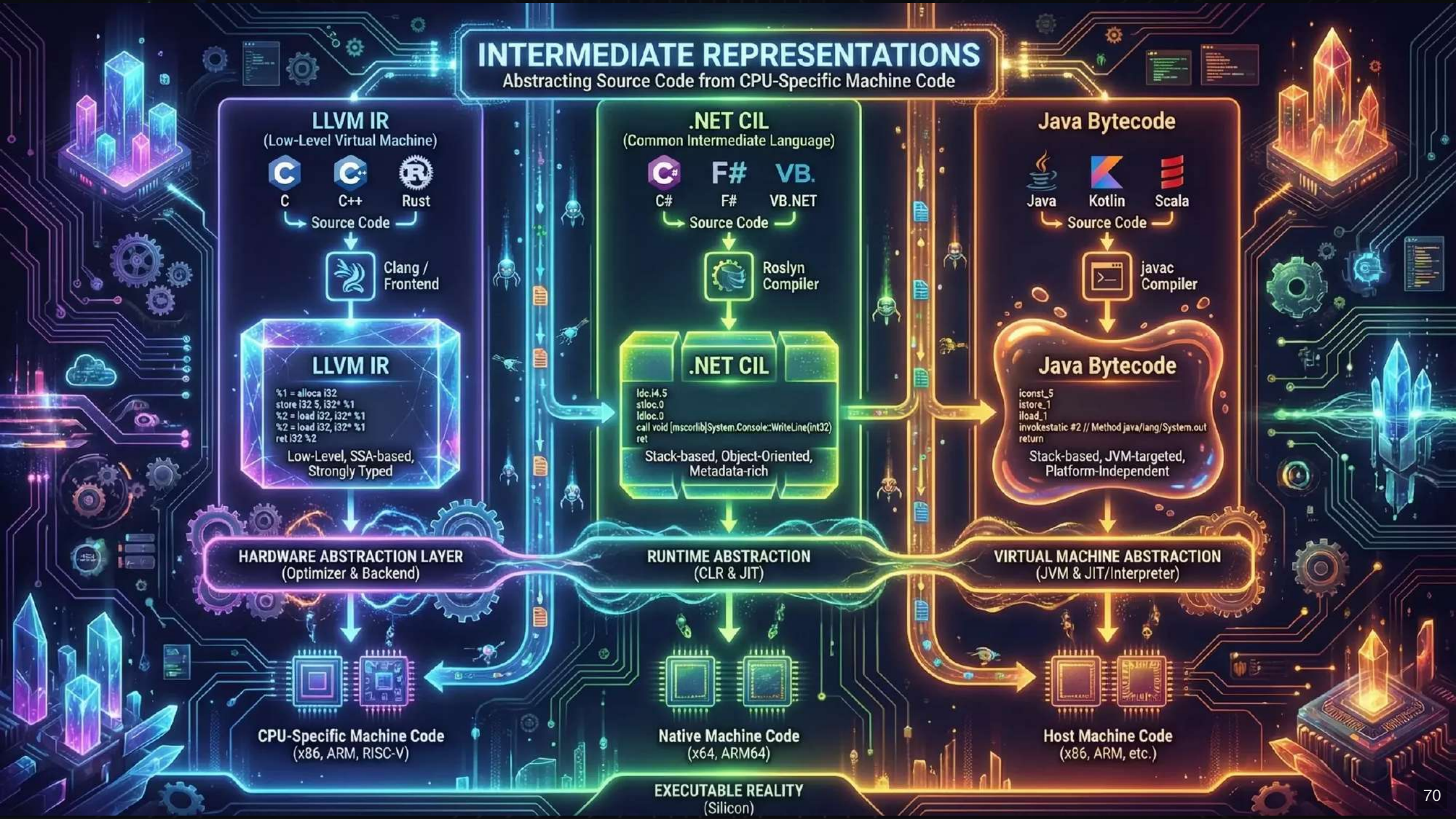


2. POWERING THE ADVANTAGE OVER JS

3. BEYOND THE BROWSER HORIZON: UNIVERSAL SCOPE



Intermediate Representations


INTERMEDIATE REPRESENTATIONS
Abstracting Source Code from CPU-Specific Machine Code
LLVM IR
(Low-Level Virtual Machine)
C
C++
Rust
Source Code
Clang / Frontend
LLVM IR
%1 = alloca i32
store i32 5, i32* %1
%2 = load i32, i32* %1
%2 = load i32, i32* %1
ret i32 %2
Low-Level, SSA-based, Strongly Typed
.NET CIL
(Common Intermediate Language)
C#
F#
VB.NET
Source Code
Roslyn Compiler
.NET CIL
ldc.i4.5
stloc.0
ldloc.0
call void [mscorlib]System.Console::WriteLine(int32)
ret
Stack-based, Object-Oriented, Metadata-rich
Java Bytecode
Java
Kotlin
Scala
Source Code
javac Compiler
Java Bytecode
iconst_5
istore_1
iload_1
invokestatic #2 // Method java/lang/System.out
return
Stack-based, JVM-targeted, Platform-Independent
HARDWARE ABSTRACTION LAYER
(Optimizer & Backend)
RUNTIME ABSTRACTION
(CLR & JIT)
VIRTUAL MACHINE ABSTRACTION
(JVM & JIT/Interpreter)
CPU-Specific Machine Code
(x86, ARM, RISC-V)
Native Machine Code
(x64, ARM64)
Host Machine Code
(x86, ARM, etc.)
EXECUTABLE REALITY
(Silicon)

LLVM IR, .NET CIL, & Java ByteCode:

A Nanopunk Comparison of Intermediate Architectures

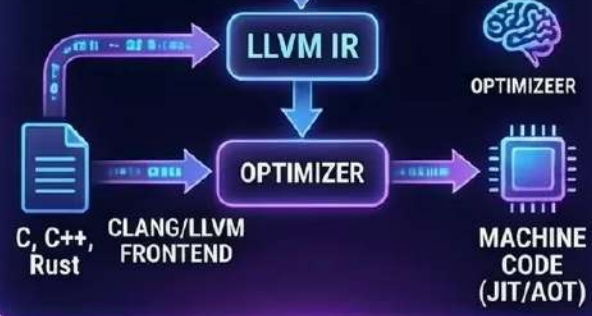
LLVM IR

(Low-Level Virtual Machine)

NATURE:
LANGUAGE-AGNOSTIC, LOW-LEVEL, SSA-BASED (STATIC SINGLE ASSIGNMENT)

```
define i32 @main() {
  %1 = alloca i32
  ...
}
```

KEY FEATURES:
MODULAR OPTIMIZATION, TARGET-INDEPENDENT, ABSTRACTION OF HARDWARE, RICH TYPE SYSTEM



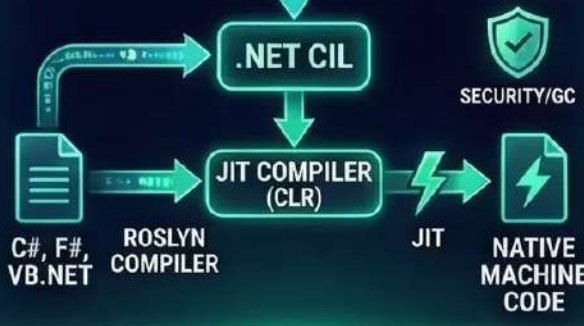
.NET CIL

(Common Intermediate Language)

NATURE:
STACK-BASED, HIGH-LEVEL, TYPE-SAFE (GC)

```
ldarg.0
ldc.i4.s 10
add
stloc.0
```

KEY FEATURES:
PLATFORM INDEPENDENCE (VIA CLR), JIT COMPILATION, RICH METADATA, GARBAGE COLLECTION, INTEROPERABILITY



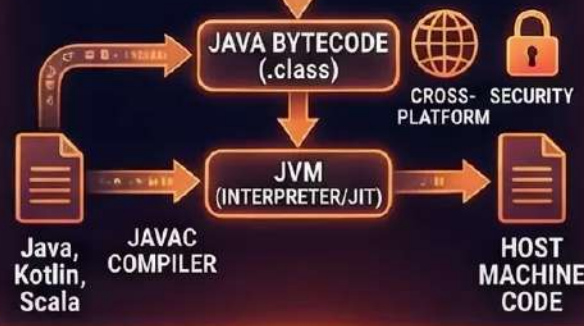
Java ByteCode

(JVM Instruction Set)

NATURE:
STACK-BASED, PLATFORM-INDEPENDENT (VIA JVM)

```
iload_1
iconst_1
iadd
istore_1
```

KEY FEATURES:
WRITE ONCE RUN ANYWHERE, DYNAMIC LINKING, SECURITY SANDBOX, AUTOMATIC MEMORY MANAGEMENT (GC)



SHARED GOAL: UNIVERSAL DIGITAL BLUEPRINTS FOR EFFICIENT, SECURE, AND PORTABLE EXECUTION IN THE NANOTECH AGE.

ASM, IR, & MACHINECODE

ASM
(ASSEMBLY LANGUAGE)

HUMAN-READABLE INSTRUCTIONS

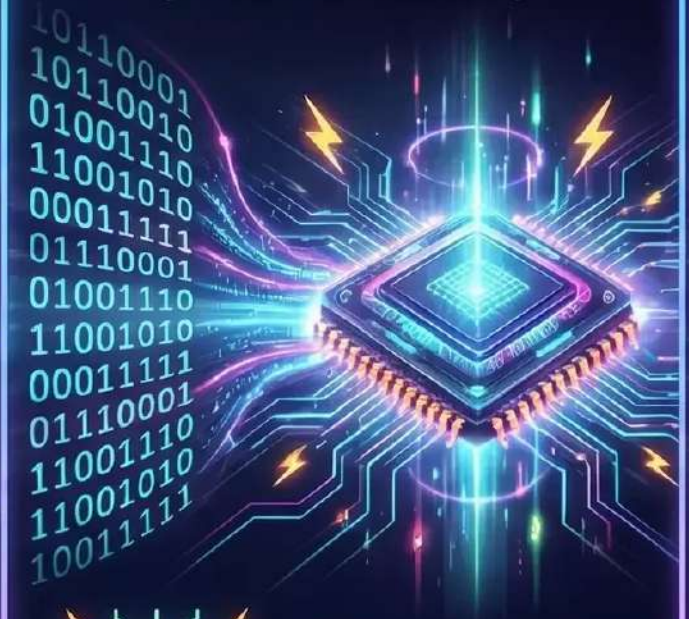
- CPU-SPECIFIC MNEMONICS
- LOW-LEVEL ABSTRACTION
- DIRECT HARDWARE CONTROL

IR
(INTERMEDIATE REPRESENTATION)

PLATFORM-AGNOSTIC HUB

- OPTIMIZATION-READY FORM
- COMPILER BRIDGE
- TARGET-INDEPENDENT CODE

MACHINECODE
(BINARY EXECUTION)



EXECUTABLE BINARY

- CPU-DIRECT INSTRUCTIONS
- BINARY FORMAT
- NATIVE EXECUTION

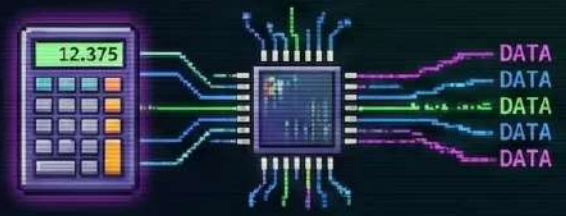
THE EVOLUTION OF CODE: FROM HUMAN-READABLE TO MACHINE-EXECUTABLE, BRIDGED BY NANOTECH OPTIMIZATION.

IEEE 754

Floating Point

FLOATING POINT:
DECODING THE DECIMAL MATRIX & IEEE 754 STANDARD

REPRESENTATION: THE BIT SLICER



DECIMAL **12.375** ⇨ SCIENTIFIC: 1.2375×10^1

BIT BREAKDOWN

- **S** (SIGN) 1-BIT
- **E** (EXPONENT) 8-BITS
- **M** (MANTISSA/FRACTION) 23-BITS

BIAS ADJ.

SIGN: +/-,
EXPONENT: SCALE,
MANTISSA: PRECISION.

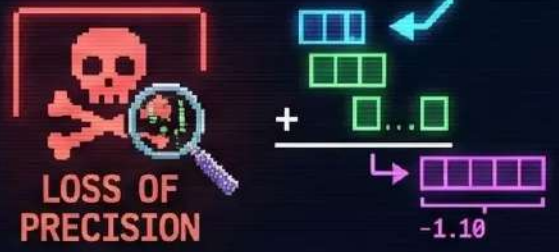
ARITHMETIC & ISSUES: THE PRECISION TRAP

NaN (NOT A NUMBER)
INF (INFINITY)

0111110001110
-001111001111

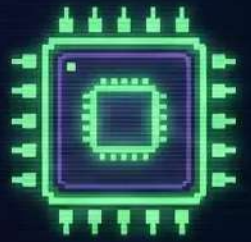
0111110001110
-001110011111

ALIGNMENT STAGE

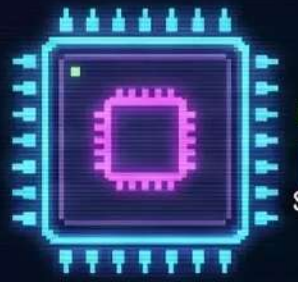


ALIGN EXPONENTS, ADD/SUB MANTISSAS, NORMALIZE.
WARNING: ROUNDING ERRORS, OVERFLOW, UNDERFLOW!

IEEE 754 STANDARD: THE UNIVERSAL PROTOCOL



32-BIT SINGLE PRECISION (FLOAT)



64-BIT DOUBLE PRECISION (DOUBLE)

DEFINES FORMATS, ROUNDING MODES, SPECIAL VALUES. THE GLOBAL LANGUAGE OF COMPUTATIONAL MATH.

```
>_ SYSTEM READY.
IEEE 754 COMPLIANT.
ACCESSING FLOATING POINT UNIT...
[OK]
```

IEEE 754:
THE FLOATING-POINT MATRIX REVEALED

32-BIT FLOAT STRUCTURE (BINARY)

+/-

SIGN (S)
1 BIT

0 = POSITIVE,
1 = NEGATIVE.
POLARITY SWITCH.

+-
/-

FLIPS THE VALUE'S DIRECTION IN THE DIGITAL VOID.

EXPONENT (E)
8 BITS

STORED AS 'BIASED' VALUE (E - 127).
MAGNITUDE SCALER.

SHIFTS THE RADIX POINT. POWER OF 2.

MANTISSA (M)
23 BITS

FRACTIONAL PART.
NORMALIZED WITH HIDDEN '1.'
PRECISION ENGINE.

DEFINES SIGNIFICANT DIGITS.
THE DATA PAYLOAD.

SPECIAL VALUES - EDGE CASES & ANOMALIES

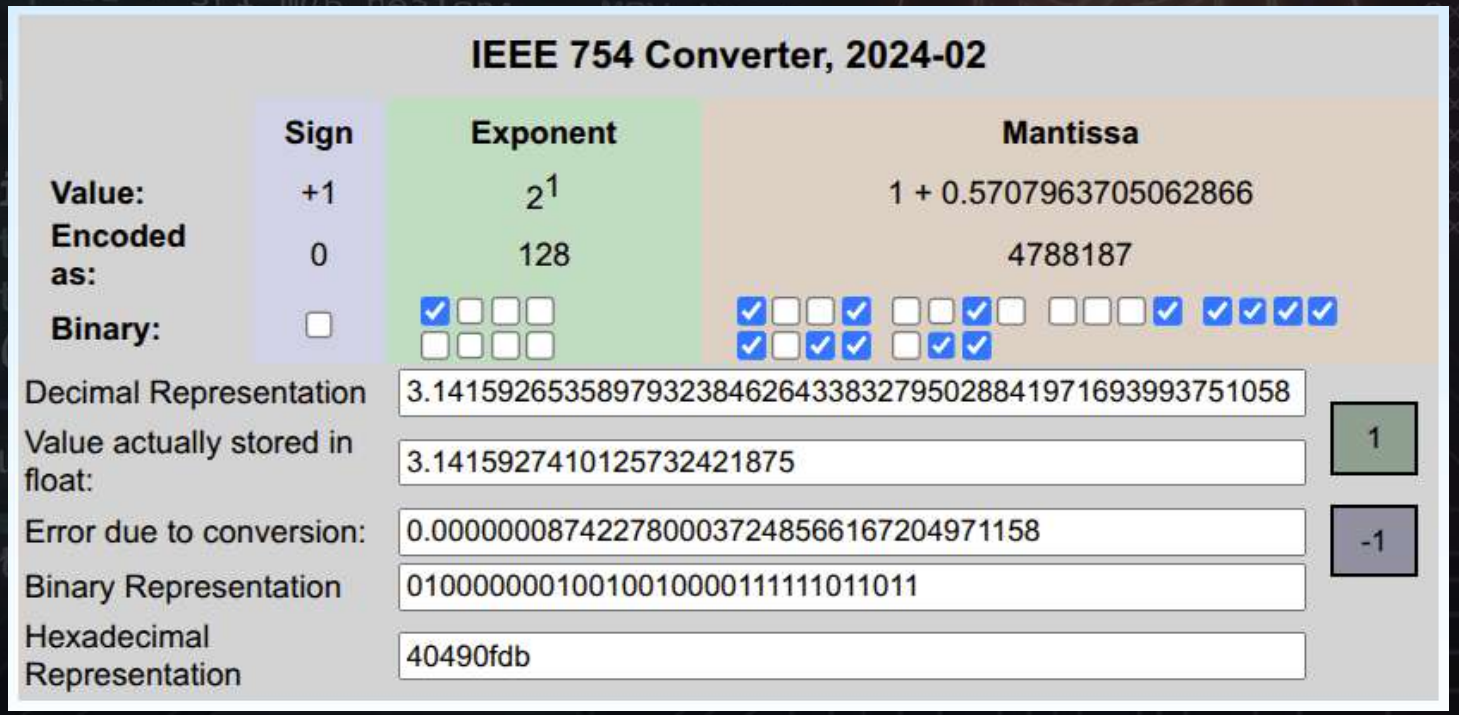
∞

EXP=255, M=0:
INFINITY (∞).
OVERFLOW BOUNDARY.

NaN

EXP=255, M≠0:
NaN (NOT A NUMBER).
CALCULATION ERROR.

>_ SYSTEM STATUS: DECODED. ACCESSING FLOATING-POINT UNIT. [EXECUTE]



IEEE 754

The standard for floating points

Established in 1985

lukaskollmer.de/ieee-754-visualizer/

h-schmidt.net/FloatConverter/

standards.ieee.org/ieee/754/6210/

en.wikipedia.org/wiki/IEEE_754

/wiki/Single-precision

/wiki/Double-precision

Title

- Bullet list